

EdgeShard: Efficient LLM Inference via Collaborative Edge Computing

Mingjin Zhang, Jiannong Cao, *Fellow, IEEE*, Xiaoming Shen, Zeyang Cui

Abstract—Large language models (LLMs) have shown great potential in natural language processing and content generation. However, current LLMs heavily rely on cloud computing, leading to prolonged latency, high bandwidth cost, and privacy concerns. Edge computing is promising to address such concerns by deploying LLMs on edge devices, closer to data sources. Some works try to leverage model quantization to reduce the model size to fit the resource-constraint edge devices, but they lead to accuracy loss. Other works use cloud-edge collaboration, suffering from unstable network connections. In this work, we leverage collaborative edge computing to facilitate the collaboration among edge devices and cloud servers for jointly performing efficient LLM inference. We propose a general framework to partition the LLM model into shards and deploy on distributed devices. To achieve efficient LLM inference, we formulate an adaptive joint device selection and model partition problem and design an efficient dynamic programming algorithm to optimize the inference latency and throughput, respectively. Experiments of Llama2 serial models on a heterogeneous physical prototype demonstrate that EdgeShard achieves up to 50% latency reduction and 2x throughput improvement over baseline methods.

Index Terms—Large Language Models, Edge Computing, Edge AI, Distributed Machine Learning.

I. INTRODUCTION

Recently, the emergence of Large Language Models (LLMs) has attracted widespread attention from the public, industry, and academia, representing a significant breakthrough in artificial intelligence (AI). Many players are coming into this field with their advanced models, such as OpenAI’s GPT-4 [1], Meta’s Llama [2], and Google’s PALM [3]. Built on the foundation of transformer architecture [4], LLMs are characterized by their massive scale in terms of the number of parameters and the amount of data they are trained on. The scale of LLMs, often numbering in hundreds of billions of parameters, enables the models to capture complex patterns in language and context, making them highly effective at generating coherent and contextually appropriate responses. Such a phenomenon is also known as “intelligence emergence”. The outstanding capability of LLMs makes them valuable and well-performed in a wide range of applications, from ChatBot and content generation (e.g., text summation and code generation) to assisting tools of education and research.

However, current LLMs heavily rely on cloud computing, suffering from long response time, high bandwidth cost, and privacy concerns [5]. Firstly, the reliance on cloud computing hampers the capability for rapid model inference necessary for

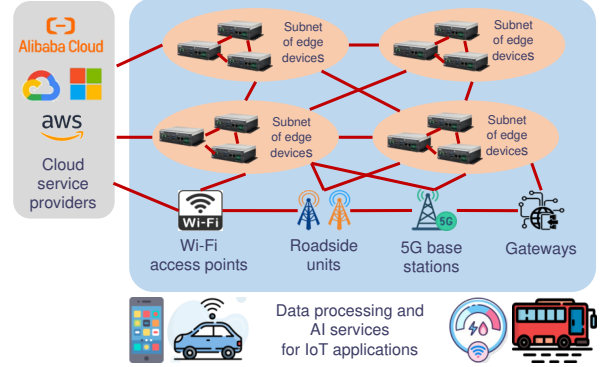


Fig. 1. Collaborative edge computing integrates the computing resources of ubiquitous geo-distributed devices for jointly performing computational tasks, with great benefits of enlarged resource pool, low-latency data processing, flexible device access, and expanded service region.

real-time applications such as robotics control, navigation, or exploration, where immediate responses are crucial. Secondly, the transmission of large amounts of data, including texts, video, images, audio, and IoT sensing data, to the cloud data centers leads to substantial bandwidth consumption and immense strain on the network architecture. Thirdly, cloud-based LLMs raise significant privacy issues, especially when handling sensitive data of hospitals and banks, as well as personal data like text inputs and photos on mobile phones.

Edge computing is a promising solution to address the aforementioned challenges by deploying LLMs on edge devices (e.g., edge servers, edge gateways, and mobile phones) at the network edge closer to the data sources [6]. However, LLMs are computation-intensive and resource-greedy. For example, the inference of a full-precision Llama2-7B model requires at least 28GB memory, which may exceed the capacity of most edge devices. Some works leverage model quantization [7]–[12] to reduce the model size to fit into the resource-constraint edge devices. However, they often lead to accuracy loss. Other works tend to use cloud-edge collaboration [13], [14], which partitions the LLMs into two sub-models and offloads part of the computation workload to the powerful cloud servers with high-end GPUs. However, the latency between edge devices and cloud servers is usually high and unstable.

Alternatively, we have witnessed the continuous growth of the computing power of edge in recent years, and a large number of edge servers and edge clouds have been deployed at the network edge, leaving significant resources to be used. Collaborative edge computing (CEC) [15], [16] is hence proposed recently to integrate the computing re-

M. Zhang, J. Cao, and X. Shen, and Z. Cui are with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong.
E-mail: {csmzhang, csjcao}@comp.polyu.edu.hk

EdgeShard: 通过协作边缘计算进行高效的 LLM 推理

张明金、曹建农、*Fellow, IEEE*, 沉晓明、崔泽洋

Abstract—大语言模型 (LLM) 在自然语言处理和内容生成方面显示出巨大的潜力。然而，目前的大语言模型严重依赖云计算，导致延迟时间长、带宽成本高和隐私问题。边缘计算有望通过在更靠近数据源的边缘设备上部署大语言模型来解决此类问题。一些工作尝试利用模型量化来减小模型大小以适应资源受限的边缘设备，但它们会导致准确性损失。其他作品采用云边协作，存在网络连接不稳定的问题。在这项工作中，我们利用协作边缘计算来促进边缘设备和云服务器之间的协作，以共同执行高效的 LLM 推理。我们提出了一个通用框架，将 LLM 模型划分为分片并部署在分布式设备上。为了实现高效的 LLM 推理，我们制定了自适应联合设备选择和模型划分问题，并设计了高效的动态编程算法来分别优化推理延迟和吞吐量。Llama2 串行模型在异构物理原型上的实验表明，与基线方法相比，EdgeShard 实现了高达 50% 的延迟减少和 2 倍的吞吐量提高。

Index Terms—大语言模型、边缘计算、边缘人工智能、分布式机器学习。

简介

近年来，大语言模型 (LLM) 的出现引起了公众、工业界和学术界的广泛关注，代表着人工智能 (AI) 的重大突破。许多玩家带着他们的先进模型进入这个领域，例如 OpenAI 的 GPT-4 [1]、Meta 的 Llama [2] 和 Google 的 PALM [3]。LLM 建立在 Transformer 架构的基础上 [4]，其特点是参数数量和训练数据量巨大。大语言模型的规模通常有数千亿个参数，使模型能够捕获语言和上下文中的复杂模式，从而使它们能够非常有效地生成连贯且适合上下文的响应。这种现象也被称为“智能涌现”。大语言模型的出色能力使其在从聊天机器人和内容生成（例如文本求和和代码生成）到教育和研究辅助工具的广泛应用中具有价值且表现良好。

然而，当前的大语言模型严重依赖云计算，存在响应时间长、带宽成本高和隐私问题等问题[5]。首先，对云计算的依赖阻碍了快速模型推理的能力。

M. Zhang, J. Cao, and X. Shen, and Z. Cui are with the Department of Computing, The Hong Kong Polytechnic University, Hong Kong.
E-mail: {csmzhang, csjcao}@comp.polyu.edu.hk

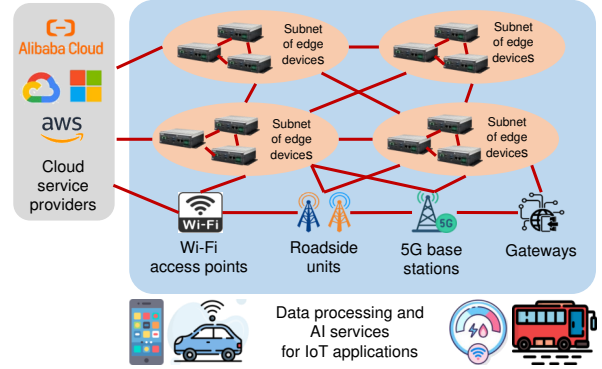


图1. 协作边缘计算整合了无处不在的地理分布式设备的计算资源，共同执行计算任务，具有扩大资源池、低延迟数据处理、灵活的设备访问和扩大服务区域的巨大好处。

实时应用程序，例如机器人控制、导航或探索，其中即时响应至关重要。其次，将大量数据（包括文本、视频、图像、音频和物联网传感数据）传输到云数据中心会导致大量的带宽消耗和网络架构的巨大压力。第三，基于云的法学硕士引发了严重的隐私问题，特别是在处理医院和银行的敏感数据以及手机上的文本输入和照片等个人数据时。

边缘计算是一种很有前途的解决方案，通过在靠近数据源的边缘设备的边缘设备（例如边缘服务器、边缘网关和移动电话）上部署 LLM 来解决上述挑战[6]。然而，法学硕士是计算密集型和资源贪婪的。例如，全精度 Llama2-7B 模型的推理需要至少 28GB 内存，这可能超出大多数边缘设备的容量。一些工作利用模型量化[7]-[12]来减小模型大小以适应资源受限的边缘设备。然而，它们常常导致准确性损失。其他工作倾向于使用云边缘协作[13]、[14]，它将 LLM 划分为两个子模型，并将部分计算工作负载卸载到具有高端 GPU 的强大云服务器上。然而，边缘设备和云服务器之间的延迟通常很高且不稳定。

另一方面，近年来我们看到边缘计算能力不断增长，大量的边缘服务器和边缘云部署在网络边缘，留下了大量的资源可供使用。因此，最近提出了协作边缘计算 (CEC) [15]、[16]来集成计算资源

sources of geo-distributed edge devices and cloud servers for efficient resource utilization and performance optimization. As shown in Fig. 1, ubiquitous and distributed edge devices and cloud servers are connected and form a shared resource pool, collaboratively providing instant data processing and AI services. CEC is different from existing edge computing research. Existing edge computing research focuses on the vertical collaboration among cloud, edge, and end devices, while neglecting horizontal edge-to-edge collaborations, suffering from unoptimized resource utilization, restricted service coverage, and uneven performance.

Motivated by the vision of CEC, we propose a general LLM inference framework, named EdgeShard, to support efficient collaborative LLM inference on distributed edge devices and cloud servers. For simplicity, we use computing devices below to refer to edge devices and cloud servers. Given a network with heterogeneous computing devices, EdgeShard partitions the LLM into multiple shards and allocates them to judicious devices based on the heterogeneous computation and networking resources, as well as the memory budget of devices. To optimize performance, we formulate a joint device selection and model partition problem and design an efficient dynamic programming algorithm to minimize the inference latency and maximize the inference throughput, respectively. Extensive experiments on a practical testbed show that EdgeShard reduces up to 50% latency and achieves 2x throughput over on-device and vertical cloud-edge collaborative inference methods.

Our work is different from those works that partition the LLMs and allocate to multiple GPUs in cloud data centers, such as Gpipe [17] and PipeDream [18]. Deploying LLM at edge computing is vastly different from that in the cloud. First, cloud servers are usually with homogeneous GPUs, while edge devices are with heterogeneous computation capabilities in nature. Second, modern cloud GPUs for LLMs are usually connected by high-bandwidth networks, such as InfiniBand and Nvlinks, while edge devices are connected with heterogeneous and low-bandwidth networks. For example, the bandwidth of NVlinks can go up to 600GB/s, while the bandwidth among edge devices ranges from dozens of Kbps to 1000Mbps. The solution of LLMs deployment designed for cloud data centers neglect the heterogeneous and resource-constrained edge computing environment.

Our contributions are three folds.

- First, we propose a general LLM inference framework for deploying LLMs in the edge computing environment, which enables the collaborative inference among heterogeneous edge devices and cloud servers.
- Further, we quantitatively study how to select computing devices and how to partition the LLM for optimized performance. We mathematically formulate a joint device selection and model partition problem, and propose a dynamic programming algorithm to optimize the latency and throughput, respectively.
- We also evaluate the performance of EdgeShard with state-of-the-art Llama2 serial models on a physical testbed. Experimental results show EdgeShard remarkably outperforms various baseline methods.

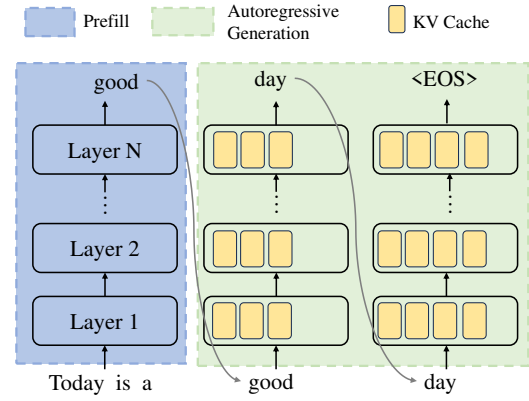


Fig. 2. LLM inference has an autoregressive nature.

TABLE I
MINIMUM MEMORY USAGE OF LLMs INFERENCE AND MEMORY CAPACITY OF EDGE DEVICES.

Model	Full Precision	8-bit	4-bit	Edge Devices
Llama2-7B	28GB	7GB	3.5GB	Smartphone(6-12GB)
Llama2-13B	52GB	13GB	6.5GB	Jetson Orin(8-16GB)
Llama2-70B	280GB	70GB	35GB	Jetson AGX(32-64GB)

II. PRELIMINARIES AND MOTIVATIONS

Generative LLM Inference. LLMs generally refer to decoder-based transformer models with billions of parameters. Different from encoder-based architecture like BERT [19], whose inference process is single phase, the process of LLM inference is iterative and typically involves two phases: the prompt processing phase and the autoregressive generation. The prompt processing phase is also known as prefill.

In the prompt processing phase, the model takes the user initial token $(x_1, \dots, x_n)$ as input and generates the first new token x_{n+1} by computing the probability $P(x_{n+1} | x_1, \dots, x_n)$.

In the autoregressive generation phase, the model generates one token at a time, based on both the initial input and the tokens it has generated so far. This phase generates tokens sequentially for multiple iterations until a stopping criterion is met, i.e., either when generating an end-of-sequence (EOS) token or reaching the maximum number of tokens specified by user or constrained by the LLM.

As shown in Fig. 2, suppose the LLM model has N layers, which will take a sequence of input tokens and run all layers to generate a token in a one-by-one manner. In the prefill phase, the model takes the input ("Today is a") at once, and the first generated token is "good." In the autoregressive generation phase, the model first takes ("Today is a good") as input and generates the next token ("day"). It then takes ("Today is a good day") as input and generates the next token ("EOS"), which indicates the end of the generation. Since a token generated is determined by all its previous token in a sequence, LLMs utilize Key-Value caching (KV caching) to avoid repetitive computation, storing past computations to expedite responses, thereby reducing computational workload and improving response times. The time to generate a token

地理分布式边缘设备和云服务器的来源，以实现高效的资源利用和性能优化。如图1所示，无处不在的分布式边缘设备和云服务器连接起来，形成共享资源池，协同提供即时数据处理和人工智能服务。CEC与现有的边缘计算研究不同。现有的边缘计算研究侧重于云、边、端设备之间的纵向协作，而忽视了边到边的横向协作，存在资源利用未优化、服务覆盖受限、性能参差不齐等问题。

受 CEC 愿景的推动，我们提出了一个通用的 LLM 推理框架，名为 EdgeShard，以支持分布式边缘设备和云服务器上的高效协作 LLM 推理。为了简单起见，我们下面使用计算设备来指代边缘设备和云服务器。给定一个具有异构计算设备的网络，EdgeShard 将 LLM 划分为多个分片，并根据异构计算和网络资源以及设备的内存预算将它们分配给明智的设备。为了优化性能，我们制定了联合设备选择和模型分区问题，并设计了高效的动态编程算法，以分别最小化推理延迟和最大化推理吞吐量。在实际测试台上进行的大量实验表明，与设备端和垂直云边协同推理方法相比，EdgeShard 减少了高达 50% 的延迟，并实现了 2 倍的吞吐量。

我们的工作不同于那些将 LLM 分区并分配给云数据中心中多个 GPU 的工作，例如 Gpipe [17] 和 PipeDream [18]。在边缘计算中部署 LLM 与在云中部署 LLM 有很大不同。首先，云服务器通常具有同构 GPU，而边缘设备本质上具有异构计算能力。其次，LLM 的现代云 GPU 通常通过高带宽网络连接，例如 InfiniBand 和 Nvlinks，而边缘设备则通过异构和低带宽网络连接。例如，Nvlinks 的带宽可以高达 600GB/s，而边缘设备之间的带宽则从几十 Kbps 到 1000 Mbps。针对云数据中心设计的 LLM 部署方案忽视了异构且资源受限的边缘计算环境。

我们的贡献有三倍。

- 首先，我们提出了一个通用的 LLM 推理框架，用于在边缘计算环境中部署 LLM，从而实现异构边缘设备和云服务器之间的协作推理。
- 此外，我们定量研究了如何选择计算设备以及如何划分 LLM 以优化性能。我们在数学上制定了联合设备选择和模型划分问题，并提出了一种动态编程算法来分别优化延迟和吞吐量。
- 我们还在物理测试台上使用最先进的 Llama2 系列模型评估了 EdgeShard 的性能。实验结果表明 EdgeShard 明显优于各种基线方法。

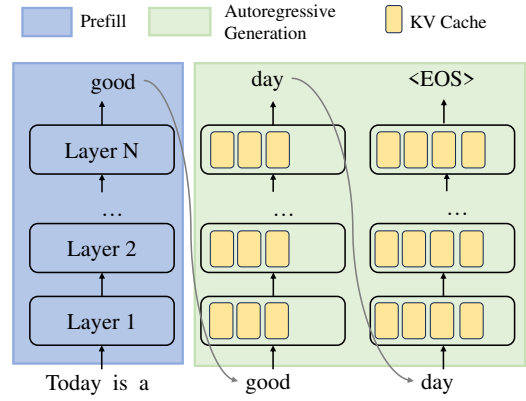


图 2. LLM 推理具有自回归性质。

表 I LLMs 推理的最低内存使用量和边缘设备的内存容量。

Model	Full Precision	8-bit	4-bit	Edge Devices
Llama2-7B	28GB	7GB	3.5GB	Smartphone(6-12GB)
Llama2-13B	52GB	13GB	6.5GB	Jetson Orin(8-16GB)
Llama2-70B	280GB	70GB	35GB	Jetson AGX(32-64GB)

二.准备工作和动机

生成大语言模型推理。LLM 通常指的是具有数十亿参数的基于解码器的变压器模型。与 BERT [19] 等基于编码器的架构的推理过程是单阶段不同，LLM 推理过程是迭代的，通常涉及两个阶段：提示处理阶段和自回归生成。提示处理阶段也称为预填充。

在提示处理阶段，模型将用户初始令牌 ($x_1, \dots, x_n$) 作为输入，并通过计算概率 $P(x_{n+1} | x_1, \dots, x_n)$ 生成第一个新令牌 x_{n+1} 。

在自回归生成阶段，模型根据初始输入和迄今为止生成的标记一次生成一个标记。此阶段为多次迭代顺序生成令牌，直到满足停止标准，即生成序列结束 (EOS) 令牌或达到用户指定或 LLM 约束的最大令牌数量时。

如图2所示，假设 LLM 模型有 N 层，它将采用一系列输入令牌并运行所有层以逐一生成令牌。在预填充阶段，模型立即接受输入 (“Today is a”)，第一个生成的标记是 “good”。在自回归生成阶段，模型首先将 (“今天很好”) 作为输入并生成下一个标记 (“day”)。然后它以 (“今天是个好日子”) 作为输入并生成下一个代币 (“<EOS>”)，这表明生成的结束。由于生成的令牌是由序列中所有先前的令牌决定的，LLM 利用键值缓存 (KV 缓存) 来避免重复计算，存储过去的计算以加快响应速度，从而减少计算工作量并缩短响应时间。生成 token 的时间

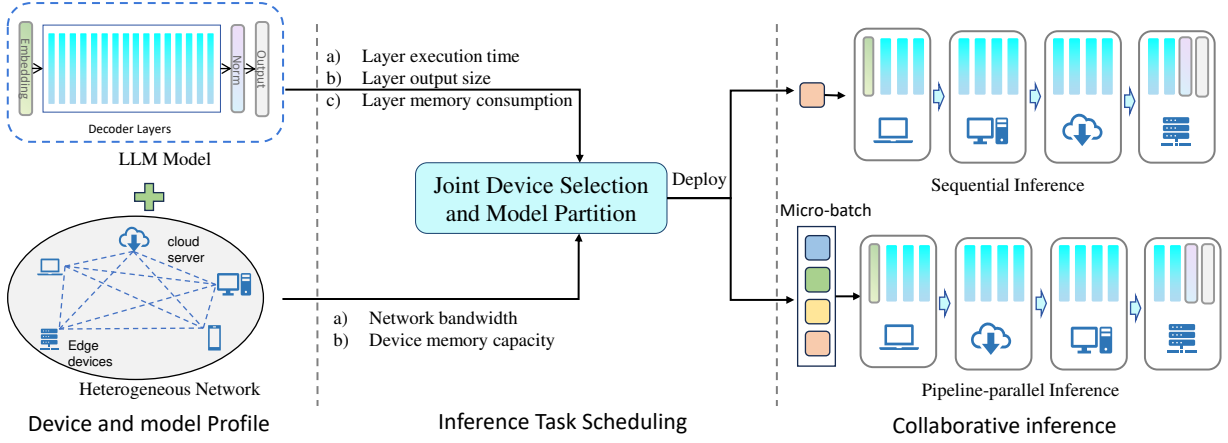


Fig. 3. Framework of EdgeLLM. It consists of three stages: offline profiling, task scheduling optimization, and online collaborative LLM inference.

in the prefill stage is much higher (usually 10x) than that of in the autoregressive stage, as the prefill stage needs to calculate the KV cache of all input tokens as initialization.

LLMs are memory-consuming. A single edge device may not have sufficient memory to accommodate a LLM model. Take one of the most popular LLM models, i.e., Llama2, as an example. As shown in Table. I, Llama2 has three different versions, i.e., 7B, 13B, and 70B. We can see from the Table that the full precision inference of Llama2-7B requires at least 28GB memory, but the smartphones usually only have 6-12 GB memory, and the Jetson Orin NX has 8-16 GB memory. They are unable to burden the on-device LLM inference. Some works try to use low-precision quantization, e.g., 8 bit and 4 bit. However, it may still exceed the memory capacity of edge devices. For example, the 4-bit inference of Llama2-70B requires at least 35GB memory, which cannot be accommodated on most edge devices. Moreover, low-precision inference leads to performance degradation.

In this work, we leverage collaborative edge computing, a computing paradigm where geo-distributed edge devices and cloud servers collaborate to perform computational tasks. Based on that idea, we propose EdgeShard, a general LLM inference framework that allows adaptive device selection and LLM partition over distributed computing devices, to address the high memory requirements and leverage heterogeneous resources to optimize LLM inference.

III. COLLABORATIVE EDGE COMPUTING FOR LLMs

There are three stages of the framework, including profiling, task scheduling optimization, and collaborative inference. The workflow is shown in Fig. 3.

Profiling is an offline step that profiles the necessary runtime traces for the optimization step and only needs to be done once. Those traces include: 1) the execution time of each layer on different devices; 2) the size of activations and memory consumption for each layer of the LLM model; 3) available memory of each device and the bandwidth among devices. For the execution time of each layer, we profile the time to generate a token in the prefill stage and autoregressive stage, respectively, and take the average. For those devices

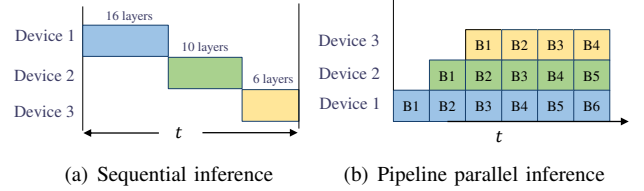


Fig. 4. Collaborative LLM inference

that may not have efficient memory to hold the full model for performing the profiling, we utilize a dynamic model loading technology, where the model layers are consecutively loaded to fit the constrained memory. The profiling information will then be used to support intelligent task scheduling strategies.

Scheduling Optimization. At the task scheduling optimization stage, the scheduler generates a deployment strategy by determining which device to participate in, how to partition the LLM model in a layer wise, and which device should the model shard be allocated to. The strategy thoroughly considers the heterogeneous resources, the memory budget of devices, and the privacy constraint, and later be applied to selected devices for efficient LLM inference. More details is described in Sec. IV.

Collaborative inference. After getting the LLM model partition and allocation strategy, the selected devices will perform the collaborative inference. We pre-allocate memory space for KV cache on each participating device. We consider two cases for the collaborative inference, i.e., sequential inference and pipeline parallel inference.

In sequential inference, devices take turns to perform the computation with the allocated model shards. As shown in Fig. 4(a), suppose the LLM model is partitioned into 3 shards and allocated to device 1, 2, and 3, respectively. Device 1 will first process the input data and then send the activations/outputs to device 2, which will process the data and then transmit to device 3. Sequential inference is suitable for serving a single user, such as in smart home scenario, where users' personal devices (e.g., tablet, phones, and smart speaker) collaborate to perform LLM inference. In such scenario, user inputs a prompt

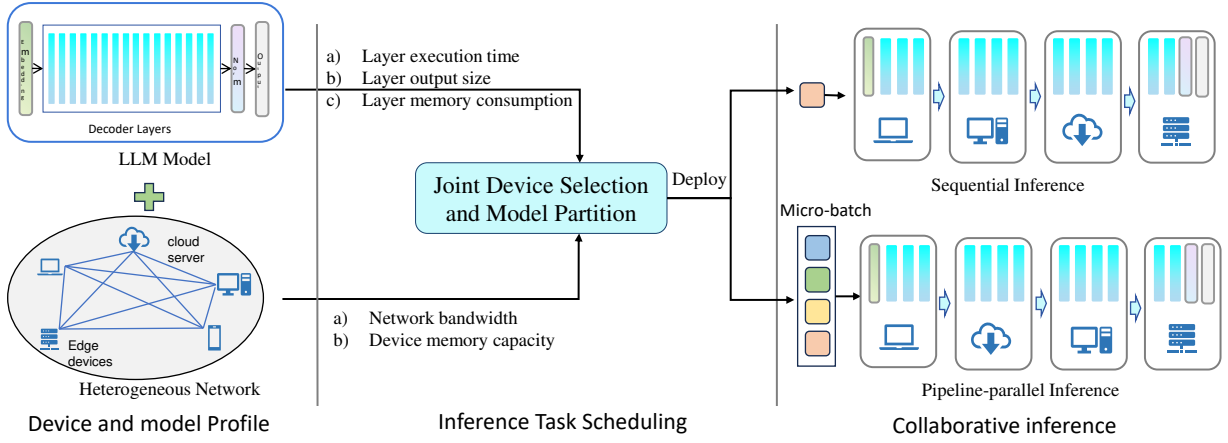


Fig. 3. EdgeLLM的框架。它由三个阶段组成：离线分析、任务调度优化和在线协作LLM推理。

预填充阶段的值比自回归阶段的值要高得多（通常是10倍），因为预填充阶段需要计算所有输入令牌的KV cache作为初始化。

LLM 非常消耗内存。单个边缘设备可能没有足够的内存来容纳 LLM 模型。以最流行的LLM模型之一Llama2为例。如表所示。《I, Llama2》有7B、13B、70B三个不同版本。从表中我们可以看出，Llama2-7B的全精度推理至少需要28GB内存，但智能手机通常只有6-12GB内存，而Jetson Orin NX有8-16GB内存。他们无法承担设备上 LLM 推理的负担。有些作品尝试使用低精度量化，例如 8 位和 4 位。然而，它仍然可能超出边缘设备的存储容量。例如，Llama2-70B的4位推理需要至少35GB内存，这在大多数边缘设备上都无法容纳。此外，低精度推理会导致性能下降。

在这项工作中，我们利用协作边缘计算，这是一种计算范式，地理分布式边缘设备和云服务器协作执行计算任务。基于这个想法，我们提出了 EdgeShard，这是一种通用的 LLM 推理框架，允许自适应设备选择和分布式计算设备上的 LLM 分区，以解决高内存需求并利用异构资源来优化 LLM 推理。

三. LLMS 的协作边缘计算

该框架分为三个阶段，包括分析、任务调度优化和协作推理。工作流程如图3所示。

分析是一个离线步骤，用于分析优化步骤所需的运行时跟踪，并且只需完成一次。这些痕迹包括：1) 不同设备上每一层的执行时间；2) LLM模型每一层的激活大小和内存消耗；3) 每个设备的可用内存以及设备间的带宽。对于每一层的执行时间，我们分别分析预填充阶段和自回归阶段生成令牌的时间，并取平均值。对于那些设备

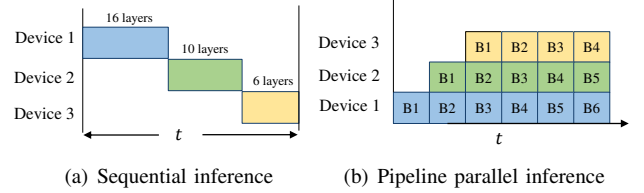


图 4. 协作 LLM 推理

由于可能没有有效的内存来保存用于执行分析的完整模型，我们利用动态模型加载技术，连续加载模型层以适应受限的内存。然后，分析信息将用于支持智能任务调度策略。

调度优化。在任务调度优化阶段，调度器通过确定参与哪个设备、如何对LLM模型进行分层划分以及模型分片应该分配到哪个设备来生成部署策略。该策略充分考虑了异构资源、设备的内存预算和隐私约束，随后应用于选定的设备以实现高效的 LLM 推理。更多细节在第 2 节中描述。

协同推理。得到LLM模型划分和分配策略后，所选设备将进行协同推理。我们在每个参与设备上为 KV 缓存预先分配内存空间。我们考虑协作推理的两种情况，即顺序推理和管道并行推理。

在顺序推理中，设备轮流使用分配的模型分片执行计算。如图4 (a) 所示，假设LLM模型被划分为3个分片，并分别分配给设备1、2和3。设备 1 将首先处理输入数据，然后将激活/输出发送到设备 2，设备 2 处理数据，然后传输到设备 3。顺序推理适合为单个用户服务，例如在智能家居场景中，用户的个人设备（例如平板电脑、手机和智能扬声器）协作执行 LLM 推理。在这种情况下，用户输入提示

TABLE II
LIST OF NOTATIONS

Symbol	Descriptions
$X_{i,j}$	binary variable, whether layer i of a model is allocated to device j
$t_{comp}^{i,j}$	computation time of layer i on device j
$t_{comp}^{i \rightarrow m,j}$	computation time of layer i to layer m on device j
$t_{comm}^{i-1,k,j}$	communication time to transmit activations of layer $i-1$ from device k to device j
$DP(i,j)$	minimal total execution time of the first i layers if layer i is allocated to device j
$g(i,S,k)$	processing time of the slowest node to process the first i layers with device set S

and gets the response and then input another prompt. We aims to minimize the latency of sequential inference.

However, sequential inference is not resource-efficient from the system's perspective. When device 1 is performing computation, device 2 and device 3 are idle. We thus take pipeline parallelism to improve resource utilization. For the pipeline parallel inference as taken in previous work Gpipe [17] and PipeDream [18] for cloud servers, the input data will first be split into micro-batch and subsequently feed into the system. As depicted in Fig. 4(b), device 1 first handles data B1 and then transmits intermediate data to device 2. After handling data B1, device 1 immediately goes to handle data B2. In such a pipeline manner, every device is busy with high system resource utilization.

IV. OPTIMIZE LLM INFERENCE

We consider a general collaborative edge network with heterogeneous devices and bandwidth connection. More specifically, given a set of heterogeneous devices connected with heterogeneous bandwidth, EdgeShard aims to select a subset of devices and partition the LLM into shards, which will be allocated to the selected devices to minimize the inference latency or maximize the throughput.

System Model. LLMs usually have a layered architecture, which consists of an embedding layer, multiple decoder layer, and an output layer. Sizes of parameters and activations (i.e., the output of a layer) vary across layers. We assume the model is with N layers. O_i represents the size of activations of layer i , $0 \leq i \leq N-1$. The memory consumption of a layer i is denoted by Req_i .

We consider a network consisting of M edge devices and cloud servers. The devices have heterogeneous computation and memory capabilities, and cloud servers are much more powerful than edge devices in terms of computation capability. The memory budget of a device j is Mem_j . The computing devices are interconnected. Bandwidth between a device k and a device j is $B_{k,j}$, $0 \leq k \leq M-1$, $0 \leq j \leq M-1$. There is a source node where the input tokens reside. Without loss of generality, we set the source node as node 0. The main notations used in this paper are shown in Table. II.

A. Optimize LLM inference latency

Problem Formulation. We use a binary variable $X_{i,j}$ to denote the LLM allocation strategy. $X_{i,j}$ equals to 1 if layer

i is allocated to node j . Otherwise, $X_{i,j}$ equals to zero. A layer will be and only be allocated to one node. Hence, we have $\sum_{j=0}^{M-1} X_{i,j} = 1, \forall i$. Let $t_{comp}^{i,j}$ denotes the computation time of layer i on node j . Suppose layer $i-1$ and layer i are allocated to node k and node j , respectively. We use $t_{comm}^{i-1,k,j}$ to denote the communication time to transmit the activations of layer $i-1$ from node k to node j . The data transmission time is determined by the output size of a layer and the bandwidth between two nodes. If layer $i-1$ and layer i are on the same node, we assume the transmission time is zero. Hence, we have

$$t_{comm}^{i-1,k,j} = \begin{cases} \frac{O_{i-1}}{B_{k,j}}, & \text{if } k \neq j \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

The total inference time can thus be calculated by the following equation.

$$T_{tol} = \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} X_{i,j} * t_{comp}^{i,j} + \sum_{i=1}^{N-1} \sum_{j=0}^{M-1} \sum_{k=0}^{M-1} X_{i-1,k} * X_{i,j} * t_{comm}^{i-1,k,j} \quad (2)$$

Hence, the problem of minimizing the LLM inference latency can be formulated as follows, where Eq. (4) is the privacy constraint. It shows that the first layer of the LLM model should always be allocated to node 0, which is set to be the source node with input tokens. In such a case, the raw input data resides on the source node and avoids to be transmitted among computing devices. Eq. (5) shows that the memory requirements of all the layers allocated to node j cannot exceed its memory budget.

$$\min T_{tol} \quad (3)$$

$$X_{0,0} = 1 \quad (4)$$

$$\sum_{i=0}^{N-1} X_{i,j} * Req_i \leq Mem_j \quad (5)$$

Solution. To minimize the inference latency, we design a dynamic programming algorithm. The intuition is that the minimal execution time of the first i layer is determined by the first $i-1$ layer, which means the optimal solution can be constructed from the optimal results of the sub-problems. It has the optimal sub-problem property, which motivates us to use dynamic programming.

Let $DP(i,j)$ denote the minimal total execution time of the first i layers after the layer i is allocated to the node j . The state transition equation is formulated as:

$$DP(i,j) = \begin{cases} \min_{\substack{k \in M \\ 1 \leq i < N-1}} (DP(i-1,k) + t_{comp}^{i,j} + t_{comm}^{i-1,k,j}) \\ \min_{\substack{k \in M \\ i=N-1}} (DP(i-1,k) + t_{comp}^{i,j} + t_{comm}^{i-1,k,j} + t_{comm}^{i,j,0}) \end{cases} \quad (6)$$

Where $DP(i-1,k)$ indicates the minimal execution time of the first $i-1$ layers if layer $i-1$ is allocated to device k . Eq. (6) shows that $DP(i,j)$ is determined by traversing at all possible nodes of the previous layer and choosing the one that

表 II 符号列表

Symbol	Descriptions
$X_{i,j}$	binary variable, whether layer i of a model is allocated to device j
$t_{comp}^{i,j}$	computation time of layer i on device j
$t_{comp}^{i \rightarrow m,j}$	computation time of layer i to layer m on device j
$t_{comm}^{i-1,k,j}$	communication time to transmit activations of layer $i-1$ from device k to device j
$DP(i,j)$	minimal total execution time of the first i layers if layer i is allocated to device j
$g(i,S,k)$	processing time of the slowest node to process the first i layers with device set S

并得到响应然后输入另一个提示。我们的目标是最大限度地减少顺序推理的延迟。

然而，从系统的角度来看，顺序推理并不节省资源。当设备1进行计算时，设备2和设备3处于空闲状态。因此，我们采用管道并行来提高资源利用率。对于云服务器的先前工作 Gpipe [17] 和 PipeDream [18] 中采用的管道并行推理，输入数据将首先被分割成微批次，然后输入到系统中。如图4(b)所示，设备1首先处理数据B1，然后将中间数据发送给设备2。处理完数据B1后，设备1立即去处理数据B2。在这种流水线方式下，每个设备都处于忙碌状态，系统资源利用率很高。

四. 优化 LLM 推理

我们考虑具有异构设备和带宽连接的通用协作边缘网络。更具体地说，给定一组与异构带宽连接的异构设备，EdgeShard 的目标是选择设备的子集并将 LLM 划分为分片，这些分片将分配给所选设备以最小化推理延迟或最大化吞吐量。

系统模型。LLM 通常具有分层架构，由嵌入层、多个解码器层和输出层组成。参数和激活的大小（即层的输出）因层而异。我们假设模型具有 N 层。 O_i 表示层 i , $0 \leq i \leq N-1$ 的激活大小。层 i 的内存消耗由 Req_i 表示。

我们考虑一个由 M 边缘设备和云服务器组成的网络。设备具有异构的计算和存储能力，云服务器的计算能力比边缘设备强大得多。设备 j 的内存预算是 Mem_j 。计算设备是互连的。设备 k 和设备 j 之间的带宽为 $B_{k,j}$, $0 \leq k \leq M-1$, $0 \leq j \leq M-1$ 。存在输入令牌所在的源节点。不失一般性，我们将源节点设置为节点0。本文使用的主要符号如表所示。二。

A. Optimize LLM inference latency

问题表述。我们使用二进制变量 $X_{i,j}$ 来表示 LLM 分配策略。如果层 $X_{i,j}$ 等于 1

i 分配给节点 j 。否则， $X_{i,j}$ 等于 0。一层将且仅分配给一个节点。因此，我们有 $\sum_{j=0}^{M-1} X_{i,j} = 1, \forall i$ 。令 $t_{comp}^{i,j}$ 表示节点 j 上层 i 的计算时间。假设层 $i-1$ 和层 i 分别分配给节点 k 和节点 j 。我们使用 $t_{comm}^{i-1,k,j}$ 表示将层 $i-1$ 的激活从节点 k 传输到节点 j 的通信时间。数据传输时间由层的输出大小和两个节点之间的带宽决定。如果层 $i-1$ 和层 i 位于同一节点上，我们假设传输时间为零。因此，我们有

$$t_{comm}^{i-1,k,j} = \begin{cases} \frac{O_{i-1}}{B_{k,j}}, & \text{if } k \neq j \\ 0, & \text{otherwise.} \end{cases} \quad (1)$$

因此，总推理时间可以通过以下等式计算。

$$T_{tol} = \sum_{i=0}^{N-1} \sum_{j=0}^{M-1} X_{i,j} * t_{comp}^{i,j} + \sum_{i=1}^{N-1} \sum_{j=0}^{M-1} \sum_{k=0}^{M-1} X_{i-1,k} * X_{i,j} * t_{comm}^{i-1,k,j} \quad (2)$$

因此，最小化 LLM 推理延迟的问题可以表述如下，其中等式：(4) 是隐私约束。它表明 LLM 模型的第一层应始终分配给节点0，该节点被设置为具有输入标记的源节点。在这种情况下，原始输入数据驻留在源节点上并且避免在计算设备之间传输。等式。(5) 表明分配给节点 j 的所有层的内存需求不能超过其内存预算。

$$\min T_{tol} \quad (3)$$

$$X_{0,0} = 1 \quad (4)$$

$$\sum_{i=0}^{N-1} X_{i,j} * Req_i \leq Mem_j \quad (5)$$

解决方案。为了最大限度地减少推理延迟，我们设计了一种动态编程算法。直觉是，第一个 i 层的最小执行时间由第一个 $i-1$ 层决定，这意味着可以根据子问题的最优结果构造最优解。它具有最优子问题的性质，这激励我们使用动态规划。

令 $DP(i,j)$ 表示层 i 分配给节点 j 之后 i 层的最小总执行时间。状态转移方程表述为：

$$DP(i,j) = \begin{cases} \min_{\substack{k \in M \\ 1 \leq k \leq N-1}} (DP(i-1,k) + t_{comp}^{i,j} + t_{comm}^{i-1,k,j}) \\ \min_{\substack{k \in M \\ i=N-1}} (DP(i-1,k) + t_{comp}^{i,j} + t_{comm}^{i-1,k,j} + t_{comm}^{i,j,0}) \end{cases} \quad (6)$$

其中 $DP(i-1,k)$ 表示如果第 $i-1$ 层分配给设备 k ，则前 $i-1$ 层的最小执行时间。等式。(6) 表明 $DP(i,j)$ 是通过遍历上一层所有可能的节点并选择其中一个来确定的

minimizes the execution time of the first i layers. Moreover, due to the autogressive nature of LLM, the generated token needs to be sent back to the source node for next iteration of generation. Hence, for the last layer $N - 1$, the communication time not only includes the data transmission time from the $N - 2$ layer, but also the transmission time to the source node $t_{comm}^{N-1,j,0}$. Additionally, we initialize $DP(0,0)$ as shown in Eq. (7) by considering the privacy constraint.

$$DP(0,0) = t_{comp}^{0,0} \quad (7)$$

By traversing each layer and each node based on Eq. (6), we can fill in the dynamic programming table $DP(i,j)$ to track the minimum total execution time to reach each layer. Finally, the minimal total execution time at the last layer can be calculated by Eq. (8). We can then get the optimal node allocation for each layer by backtracking $DP(i,j)$.

$$\min_{j=0,\dots,M-1}(DP(N-1,j)) \quad (8)$$

This method is simple and effective. With dynamic programming, we can quickly traverse the solution space and find the best LLM partition and allocation strategy. The algorithm to find the optimal LLM partition and allocation strategy for minimizing inference latency is shown in Algo. 1.

In Algo. 1, we first initialize the dynamic programming table $DP(i,j)$ and choice table $choice(i,j)$ (lines 1-2). We initialize $DP(0,0)$ according to Eq. (7). The $choice(i,j)$ records the node k to host the $i - 1$ layer. It is the optimal variable of Eq. (6). We then traverse the layers of the large language model from layer 1. For each layer, we traverse all the computing devices with sufficient memory and calculate the inference time (lines 3-19). After filling the DP table, we can find the minimal $DP(N-1,j)$, which represents the minimal time for executing the LLM model, and the last node to host layer $N - 1$. Finally, by backtracing $choice(i,j)$, we get the model partition and allocation strategy R (lines 20-28). The computational complexity of Algo. 1 is $O(N \times M \times M)$, where N is the number of layers of the LLM model and M is the number of devices in the network.

B. Optimize LLM inference throughput

Problem Formulation. For optimizing throughput, pipeline parallelism is adopted to avoid device idleness. As illustrated before, the computation time of layer i on node j is $t_{comp}^{i,j}$, and if layer i to layer m are all allocated to node j , the computation time is indicated by $t_{comp}^{i \rightarrow m,j}$. The data transmission time of the activations of layer $i - 1$ from node k to node j is $t_{comm}^{i-1,k,j}$. In pipeline parallel inference, the computation time and communication time can be overlapped to maximize the throughput. Thus, for the inference task, the maximum latency for the device j can be calculated as:

$$T_{latency}^j = \max \left\{ \begin{array}{l} t_{comp}^{i \rightarrow m,j} \\ t_{comm}^{i-1,k,j} \end{array} \right. \quad (9)$$

Ideally, for the selected devices, achieving the maximal throughput is equivalent to minimizing the latency of the slowest device. We use S to denote the selected devices, and

Algorithm 1: Joint device selection and LLM partition for optimizing latency

Input: A LLM model; Computing device M ; Profiled traces; bandwidth $B_{k,j}$;

Output: the device selection and LLM partition strategy

```
// initialization
1 Initialize DP table  $DP(i,j) = INF$ , and choice table  $choice(i,j) = NULL$  to record the strategy;
2 Enforce first layer to be allocated to node 0 by  $DP(0,0) = t_{comp}^{0,0}$  and  $choice(0,0) = 0$ ;
// fill in the DP table
3 for  $i = 1$  to  $N - 1$  do
4   for  $j = 0$  to  $M - 1$  do
5     if  $Mem_j \leq Req_i$  then
6       Continue;
7     end
8     else
9       for  $k = 0$  to  $M - 1$  do
10        Calculate the total execution time by Eq. (6) and assign it to  $t_{total}$ ;
11        if  $t_{total} \leq DP(i,j)$  then
12          Update  $DP(i,j)$  by assigning  $DP(i,j) = t_{total}$ ;
13          Update memory  $Mem_j$ ;
14          Record allocation plan  $choice(i,j) = k$ ;
15        end
16      end
17    end
18  end
19 end
// backtrace for allocation strategy
20 Initialize optimal strategy  $R$ ;
21 Find the last selected node  $N_{last} = argmin_j(DP(N-1,j))$ ;
22 Add  $N_{last}$  to  $R$ ;
23 for  $i = N - 1$  to 0 do
24   Find the previous node  $N_{last} = choice(i, N_{last})$ ;
25   Add  $N_{last}$  to  $R$ ;
26 end
27 Reverse  $R$ ;
28 return  $R$ ;
```

then the problem of maximizing the inference throughput can thus be formulated as follows, where $j \in S$.

$$\min\{T_{latency}^j | j \in S\} \quad (10)$$

Solution. Similar to minimizing the inference latency, the problem of maximizing the throughput also has an optimal sub-problem property. Maximizing the throughput of the first i layer can be deduced from solving the problem of allocating the first $i - 1$ layer, which indicates that the optimal solution of the whole problem can be constructed from the sub-problems. We also use dynamic programming to solve the problem.

最大限度地减少前 i 层的执行时间。此外，由于LLM的自渐进性质，生成的令牌需要被发送回源节点以进行下一次迭代生成。因此，对于最后一层 $N-1$ ，通信时间不仅包括来自 $N-2$ 层的数据传输时间，还包括到源节点 $t_{comm}^{N-1,j,0}$ 的传输时间。此外，我们初始化 $DP(0,0)$ ，如式 (1) 所示。(7)考虑隐私约束。

$$DP(0,0) = t_{comp}^{0,0} \quad (7)$$

根据式 (1) 遍历每一层和每个节点。(6)，我们可以填写动态规划表 $DP(i,j)$ 来跟踪到达每一层的最小总执行时间。最后，最后一层的最小总执行时间可以通过等式计算：(8)。然后我们可以通过回溯 $DP(i,j)$ 得到每层的最优节点分配。

$$\min_{j=0,\dots,M-1}(DP(N-1,j)) \quad (8)$$

此方法简单有效。通过动态规划，我们可以快速遍历解空间并找到最佳的LLM划分和分配策略。Algo 中显示了寻找最佳 LLM 分区和分配策略以最小化推理延迟的算法。1.

在阿尔戈. 1、我们首先初始化动态规划表 $DP(i,j)$ 和选择表 $choice(i,j)$ (第1-2行)。我们根据等式初始化 $DP(0,0)$ 。(7)。 $choice(i,j)$ 记录节点 k 以托管 $i-1$ 层。它是方程的最优变量。(6)。然后我们从第 1 层开始遍历大语言模型的各层。对于每一层，我们遍历所有具有足够内存的计算设备并计算推理时间 (第 3-19 行)。填充DP表后，我们可以找到最小的 $DP(N-1,j)$ ，它代表执行LLM模型的最短时间，以及托管层 $N-1$ 的最后一个节点。最后，通过回溯 $choice(i,j)$ ，我们得到模型划分和分配策略 R (第20-28行)。Algo 的计算复杂度。1 是 $O(N \times M \times M)$ ，其中 N 是 LLM 模型的层数， M 是网络中的设备数量。

B. Optimize LLM inference throughput

问题表述。为了优化吞吐量，采用管道并行以避免设备空闲。如前所述，节点 j 上的层 i 的计算时间为 $t_{comp}^{i,j}$ ，如果层 i 到层 m 全部分配给节点 j ，则计算时间由 $t_{comp}^{i \rightarrow m,j}$ 指示。从节点 k 到节点 j 的层 $i-1$ 激活的数据传输时间是 $t_{comm}^{i-1,k,j}$ 。在管道并行推理中，计算时间和通信时间可以重叠以最大化吞吐量。因此，对于推理任务，设备 j 的最大延迟可以计算为：

$$T_{latency}^j = \max \left\{ \begin{array}{l} t_{comp}^{i \rightarrow m,j} \\ t_{comm}^{i-1,k,j} \end{array} \right. \quad (9)$$

理想情况下，对于所选设备，实现最大吞吐量相当于最小化最慢设备的延迟。我们使用 S 来表示所选设备，并且

Algorithm 1: Joint device selection and LLM partition for optimizing latency

Input: A LLM model; Computing device M ; Profiled traces; bandwidth $B_{k,j}$;

Output: the device selection and LLM partition strategy

```
// initialization
1 Initialize DP table  $DP(i,j) = INF$ , and choice table  $choice(i,j) = NULL$  to record the strategy;
2 Enforce first layer to be allocated to node 0 by  $DP(0,0) = t_{comp}^{0,0}$  and  $choice(0,0) = 0$ ;
// fill in the DP table
3 for  $i = 1$  to  $N-1$  do
4   for  $j = 0$  to  $M-1$  do
5     if  $Mem_j \leq Req_i$  then
6       Continue;
7     end
8     else
9       for  $k = 0$  to  $M-1$  do
10        Calculate the total execution time by Eq. (6) and assign it to  $t_{total}$ ;
11        if  $t_{total} \leq DP(i,j)$  then
12          Update  $DP(i,j)$  by assigning  $DP(i,j) = t_{total}$ ;
13          Update memory  $Mem_j$ ;
14          Record allocation plan  $choice(i,j) = k$ ;
15        end
16      end
17    end
18  end
19 end
// backtrack for allocation strategy
20 Initialize optimal strategy  $R$ ;
21 Find the last selected node  $N_{last} = \operatorname{argmin}_j(DP(N-1,j))$ ;
22 Add  $N_{last}$  to  $R$ ;
23 for  $i = N-1$  to 0 do
24   Find the previous node  $N_{last} = choice(i, N_{last})$ ;
25   Add  $N_{last}$  to  $R$ ;
26 end
27 Reverse  $R$ ;
28 return  $R$ ;
```

那么最大化推理吞吐量的问题可以表述如下，其中 $j \in S$ 。

$$\min\{T_{latency}^j | j \in S\} \quad (10)$$

解决方案。与最小化推理延迟类似，最大化吞吐量的问题也具有最优子问题属性。通过解决第一个 $i-1$ 层的分配问题可以推导出最大化第一个 i 层的吞吐量，这表明整个问题的最优解可以从子问题构造出来。我们还使用动态规划来解决这个问题。

Let $g(i, S, k)$ denote the minimum time to process the first i layers with the set of used devices S , and the device k is the last node to be used, $k \in S$. We use $g(m, S', j)$ to denote the next state to process the first m layers with the set of used devices S' , and the device j is the last node to be used, where $0 \leq i < m \leq N - 1$, $j \in M \setminus S$, $S' = S \cup \{j\}$.

The state transition equation is formulated in Eq. (11), where $g(m, S', j)$ is determined by the previous state $g(i, S, k)$, and the maximum latency of device j , i.e., the computation time $t_{comm}^{i-1,k,j}$ and the communication time $t_{comp}^{i \rightarrow m,j}$. The final optimal solution T_{throu}^{opt} is the minimum $g(N - 1, S', j)$, where $S' \subseteq M$.

$$g(m, S', j) = \min_{S' = S \cup \{j\}} \max_{\substack{0 \leq i < m \leq N-1 \\ j \in M \setminus S}} \begin{cases} g(i, S, k) \\ t_{comm}^{i-1,k,j} \\ t_{comp}^{i \rightarrow m,j} \end{cases} \quad (11)$$

Additionally, we have constraints when performing state transition. They are the memory constraint shown in Eq. (12) and privacy constraint in Eq. (13).

$$Req_{i \rightarrow m} \leq Mem_j \quad (12)$$

$$g(1, 1, 0) = t_{comp}^{0,0} \quad (13)$$

Algo. 2 describes the pseudo-code to find the optimal solution T_{throu}^{opt} and the corresponding model partition and allocation strategy. In Algo. 2, we first initialize the dynamic programming table $g(m, S', j)$ and choice table $choice(m, S, j)$, and assign $t_{comp}^{0,0}$ to $g(1, 1, 0)$ (lines 1-2). We then traverse the layers of the large language model from layer 1. For each layer, we traverse all the computing devices with sufficient memory and calculate the maximum latency (lines 3-23). After filling the DP table, we can find the maximum latency, based on which we then backtrace the choice table and finally get the model partition and allocation strategy (lines 24-32). The computational complexity of Algo. 2 is $O(N^2 \times 2^M \times M^2)$, where N is the number of layers of the LLM model and M is the number devices in the network.

Pipeline Execution Optimization. Note that the above problem formulation and solution are based on the ideal case, where there is no idle device at any time. A device processes a batch of data and continues to handle another batch of data without waiting. However, it is impractical for LLM inference in real-world cases.

As shown in Fig. 5(a), different from those one-phase computation applications, the decoder-based LLM application has an autoregressive nature, where there will be multiple tokens to be generated and the calculation of the current token relies on all the previous tokens. The computation of the current token cannot start until it gets the previously generated token. It leads to bubbles in pipeline execution.

To approximate the ideal case and enhance the resource utilization for improving throughput, we tend to reduce the bubbles in the pipeline execution. We propose EdgeShard-No-bubbles, which allows for immediate token generation without waiting for the ending of all micro-batches in an iteration. As shown in Fig. 5(b), after the prefill stage $P1$ ends of the first batch, Device 1 immediately executes the token generation of

Algorithm 2: Joint device selection and LLM partition for optimizing throughput

Input: A LLM model; Computing devices M ; Profiled traces; bandwidth $B_{k,j}$;

Output: the device selection and LLM partition strategy R

```
// initialization
1 Initialize DP table  $g(i, S, k) = INF$ , and choice table
   $choice(m, S, j) = NULL$  to record the strategy;
2 Enforce first layer to be allocated to node 0 by
   $g(1, 1, 0) = t_{comp}^{0,0}$  and  $choice(1, 1, 0) = (0, 0, 0)$ ;
// fill in DP table
3 for  $i = 1$  to  $N - 1$  do
4   for each subset  $S \subseteq M$  do
5     for last node  $k \in S$  do
6       for  $m = i + 1$  to  $N - 1$  do
7         for  $j \in M \setminus S$  do
8           if  $Mem_j \leq \sum_i^m Req_i$  then
9             Continue;
10          end
11          else
12            Get  $S'$  by adding node  $j$  to the
              selected device set  $S$ ;
13            Calculate current maximum
              execution time  $T_{max}$  via
              Eq. (11) for the maximum
              execution time in all stages;
14          end
15          if  $T_{max} \leq g(i, S, k)$  then
16             $g(m, S', j) = T_{max}$ ;
17            Record the current strategy
               $choice(m, S', j) = (i, j, k)$ ;
18          end
19        end
20      end
21    end
22  end
23 end

// backtrace for optimal allocation
24 Initialize optimal strategy  $R$ ;
25 Find selected device set  $S$  and the last selected node
   $N_{last}$  by  $S, N_{last} = \operatorname{argmin}_{S,k} (g(N - 1, S, k))$ ;
26 Initialize  $layer = N - 1$ ;
27 while  $layer > 0$  do
28    $(i, j, k) = choice(layer, S, N_{last})$ ;
29   Add  $(i \rightarrow layer, j)$  to  $R$ ;
30   Update  $layer, S$  and  $N_{last}$ ;
31 end
32 return  $R$ ;
```

the first batch as indicated by G_{1A} . Similarly, when G_{1A} ends, Device 1 goes to the next iteration of token generation indicated by G_{1B} . Compared to EdgeShard-Bubbles, EdgeShard-No-bubbles reduces bubbles by mitigating device idle time and is expected to improve throughput. From the pipeline execution graph in Fig. 5, we can see that EdgeShard-No-

令 $g(i, S, k)$ 表示使用一组已使用的设备 S 处理前 i 层的最短时间，设备 k 是要使用的最后一个节点 $k \in S$ 。我们使用 $g(m, S', j)$ 表示下一个状态，以使用一组已使用的设备 S' 处理前 m 层，设备 j 是要使用的最后一个节点，其中 $0 \leq i < m \leq N-1$, $j \in M \setminus S$, $S' = S \cup \{j\}$ 。

状态转移方程如式 (11) 所示。 (11)，其中 $g(m, S', j)$ 由先前状态 $g(i, S, k)$ 和设备 k 的最大延迟 j 确定，即计算时间 $t_{comm}^{i-1,k,j}$ 和通信时间 $t_{comp}^{i \rightarrow m,j}$ 。最终最优解 T_{throu}^{opt} 是最小的 $g(N-1, S', j)$ ，其中 $S' \subseteq M$ 。

$$g(m, S', j) = \min_{S' = S \cup \{j\}} \max_{\substack{0 \leq i < m \leq N-1 \\ j \in M \setminus S}} \begin{cases} g(i, S, k) \\ t_{comm}^{i-1,k,j} \\ t_{comp}^{i \rightarrow m,j} \end{cases} \quad (11)$$

此外，我们在执行状态转换时也有限制。它们是方程式中所示的内存约束。 (12) 和等式中的隐私约束 (13)。

$$Req_{i \rightarrow m} \leq Mem_j \quad (12)$$

$$g(1, 1, 0) = t_{comp}^{0,0} \quad (13)$$

算法。图2描述了寻找最优解 T_{throu}^{opt} 的伪代码以及相应的模型划分和分配策略。在阿尔戈. 2，我们首先初始化动态编程表 $g(m, S', j)$ 和选择表 $choice(m, S, j)$ ，并将 $t_{comp}^{0,0}$ 分配给 $g(1, 1, 0)$ (第1-2行)。然后我们从第 1 层开始遍历大语言模型的各层。对于每一层，我们遍历所有具有足够内存的计算设备并计算最大延迟 (第 3-23 行)。填充DP表后，我们可以找到最大延迟，然后根据该最大延迟回溯选择表，最终得到模型分区和分配策略 (第24-32行)。Algo 的计算复杂度。2 是 $O(N^2 \times 2^M \times M^2)$ ，其中 N 是 LLM 模型的层数， M 是网络中的设备数量。

管道执行优化。请注意，上述问题的表述和解决方案是基于理想情况，即任何时候都没有空闲设备。设备处理完一批数据后，无需等待就可以继续处理另一批数据。然而，在现实案例中，LLM 推理是不切实际的。

如图5 (a) 所示，与那些单阶段计算应用程序不同，基于解码器的LLM应用程序具有自回归性质，其中将生成多个令牌，并且当前令牌的计算依赖于所有先前的令牌。在获得先前生成的令牌之前，当前令牌的计算无法开始。它会导致管道执行中出现气泡。

为了接近理想情况并提高资源利用率以提高吞吐量，我们倾向于减少管道执行中的气泡。我们提出 EdgeShard-No-bubbles，它允许立即生成令牌，而无需等待迭代中所有微批次的结束。如图5(b)所示，第一批预填充阶段 P 结束后，设备1立即执行令牌生成

Algorithm 2: Joint device selection and LLM partition for optimizing throughput

Input: A LLM model; Computing devices M ; Profiled traces; bandwidth $B_{k,j}$;
Output: the device selection and LLM partition strategy R

```

// initialization
1 Initialize DP table  $g(i, S, k) = INF$ , and choice table  $choice(m, S, j) = NULL$  to record the strategy;
2 Enforce first layer to be allocated to node 0 by  $g(1, 1, 0) = t_{comp}^{0,0}$  and  $choice(1, 1, 0) = (0, 0, 0)$ ;
// fill in DP table
3 for  $i = 1$  to  $N-1$  do
4   for each subset  $S \subseteq M$  do
5     for last node  $k \in S$  do
6       for  $m = i+1$  to  $N-1$  do
7         for  $j \in M \setminus S$  do
8           if  $Mem_j \leq \sum_i^m Req_i$  then
9             Continue;
10          end
11         else
12           Get  $S'$  by adding node  $j$  to the selected device set  $S$ ;
13           Calculate current maximum execution time  $T_{max}$  via Eq. (11) for the maximum execution time in all stages;
14         end
15         if  $T_{max} \leq g(i, S, k)$  then
16            $g(m, S', j) = T_{max}$ ;
17           Record the current strategy  $choice(m, S', j) = (i, j, k)$ ;
18         end
19       end
20     end
21   end
22 end
23 end
// 最优分配回溯
24 初始化最优策略  $R$ ;
25 通过  $S, N_{last} = \operatorname{argmin}_{S,k} (g(N-1, S, k))$  查找所选设备集  $S$  和最后选择的节点  $N_{last}$ ;
26 初始化  $layer = N-1$ ;
27 而  $layer > 0$  执行
28    $(i, j, k) = choice(layer, S, N_{last})$ ;
29 将  $(i, j, k)$  添加到  $R$ ;
30 更新  $layer, S$  和  $N_{last}$ ;
31 结束
32 返回  $R$ ;

```

第一批如 G_{1A} 所示。类似地，当 G_{1A} 结束时，设备1进入由 G_{1B} 指示的下一批令牌生成迭代。与 EdgeShard-Bubbles 相比，EdgeShard-No-bubbles 通过减少设备空闲时间来减少气泡，并有望提高吞吐量。从图5的管道执行图可以看出，EdgeShard-No-

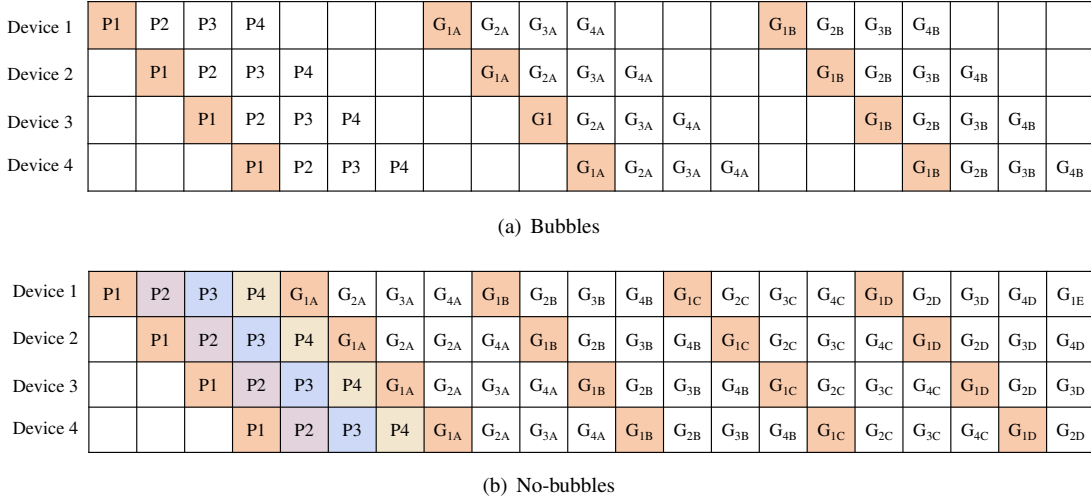


Fig. 5. Different pipeline execution strategies of EdgeShard. EdgeShard-No-bubbles reduces device idle time to improve throughput by allowing immediate token generation of a micro-batch without waiting for other micro-batches.



Fig. 6. Our testbed has heterogeneous edge devices and cloud server. Their specifications are shown in Table III.

TABLE III
SPECIFICATIONS OF HETEROGENEOUS PHYSICAL DEVICES

Category	Device	Memory	AI Performance
Edge Device	Jetson AGX Orin	32GB	3.33 TFLOPS
Edge Device	Jetson Orin NX	16GB	1.88 TFLOPS
Cloud Server	RTX 3090	24GB	36 TFLOPS

bubbles generates more tokens at the same time.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

Testbed. We use various edge devices and cloud servers to act as the heterogeneous computation devices in collaborative edge computing. The specifications of those devices are listed in Table. III. We use 15 devices, including 12 Jetson AGX Orin, 2 Jetson Orin NX, and one cloud server to configure the collaborative edge network. The physical testbed is shown in Fig. 6. Those devices are connected with a route and a switch. The bandwidth between any two devices is 1000Mbps. We use the Linux TC tool [20] to vary network bandwidth and communication latency between devices.

Benchmarks. We test the performance of EdgeShard with a series of Llama2 models [2], including Llama2-7B, Llama2-13B, and Llama2-70B. Llama2 is released by Meta in July

2023 and is one of the most popular and powerful open-source large language models, representing a groundbreaking leap in the field of artificial intelligence and natural language processing. For the model inference, we adopt the text generation task to test the performance. We use the WikiText-2 dataset [21] from HuggingFace. We extract a subset of samples with the length of input tokens as 32 and generate 96 tokens. We use full-precision model inference in all the following experiments.

Baselines. We compare the performance in terms of latency and throughput of EdgeShard with various baselines. (We don't use the cloud-only as a baseline because it requires the input token to be transmitted to the cloud server, which may lead to privacy concerns).

- **Edge-Solo.** In this case, the LLMs are deployed locally on an edge device without model partition.
- **Cloud-Edge-Even.** In this case, the LLMs are evenly partitioned into two parts. One is allocated to the edge device, and another is allocated to the cloud server.
- **Cloud-Edge-Opt.** In this case, the LLMs are partitioned into two shards. One is allocated to the edge device, and another is allocated to the cloud server. For the partition strategy of LLMs, we also use the proposed dynamic programming algorithms. The difference is that there is only two devices as the algorithm input.

B. Overall Evaluation

We set AGX Orin as the source node and the bandwidth between the source node and the cloud server as 1Mbps. The bandwidth between other computing devices is set to be 50Mbps with a variance of 20%. To test the throughput, we set the batch size as the maximum batch size that the participating devices can support. The latency and throughput of LLM inference are shown in Table. IV.

We have the following observations. First, EdgeShard is potential and beneficial for large language model deployment. For Llama2-70B model, the memory requirement is about

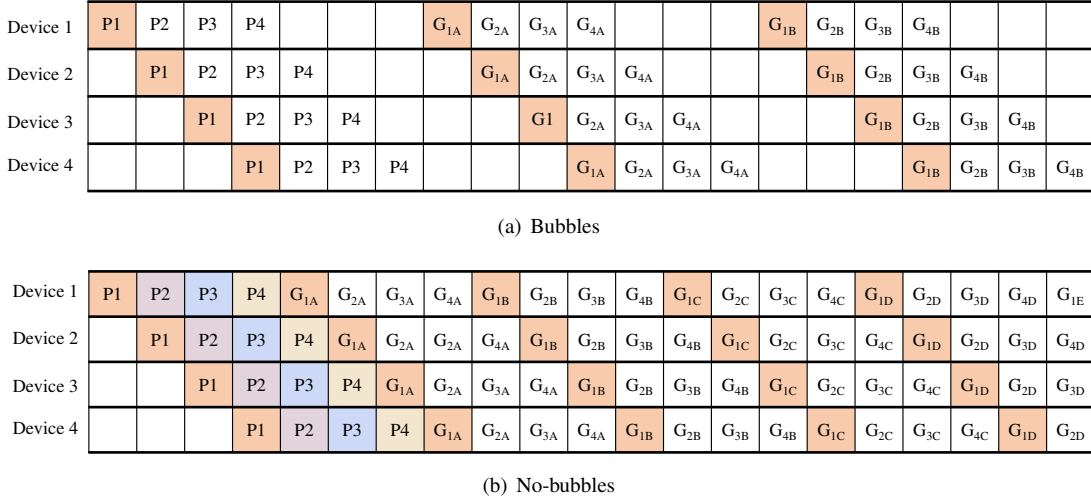


图5。EdgeShard的不同管道执行策略。EdgeShard-No-bubbles 允许立即进行操作，从而减少设备空闲时间以提高吞吐量。代币生成微批次而无需等待其他微批次。



图6.我们的测试平台具有异构边缘设备和云服务器。它们的规格如表III所示。

表 III 异构物理设备的规格

Category	Device	Memory	AI Performance
Edge Device	Jetson AGX Orin	32GB	3.33 TFLOPS
Edge Device	Jetson Orin NX	16GB	1.88 TFLOPS
Cloud Server	RTX 3090	24GB	36 TFLOPS

气泡同时产生更多代币。

五、实验评价

A. Experimental Setup

试验台。我们使用各种边缘设备和云服务器作为协作边缘计算中的异构计算设备。这些设备的规格列于表中。三. 我们使用 15 台设备（包括 12 台 Jetson AGX Orin、2 台 Jetson Orin NX）和一台云服务器来配置协作边缘网络。物理测试平台如图6所示。这些设备通过路由和交换机连接。任意两个设备之间的带宽为1000Mbps。我们使用 Linux TC 工具 [20] 来改变设备之间的网络带宽和通信延迟。

基准。我们使用一系列 Llama2 模型 [2] 测试 EdgeShard 的性能，包括 Llama2-7B、Llama2-13B 和 Llama2-70B。Llama2 于 7 月由 Meta 发布

2023年，它是最流行、最强大的开源大语言模型之一，代表着人工智能和自然语言处理领域的突破性飞跃。对于模型推理，我们采用文本生成任务来测试性能。我们使用 HuggingFace 的 WikiText-2 数据集 [21]。我们提取输入标记长度为 32 的样本子集并生成 96 个标记。我们在以下所有实验中都使用全精度模型推理。

基线。我们将 EdgeShard 的延迟和吞吐量方面的性能与各种基准进行了比较。（我们不使用仅云作为基线，因为它需要将输入令牌传输到云服务器，这可能会导致隐私问题）。

- 边缘独奏。在这种情况下，LLM 本地部署在边缘设备上，没有模型分区。
- 云-边缘-均匀。在这种情况下，大语言模型被均匀地分为两部分。一个分配给边缘设备，另一个分配给云服务器。
- 云-边缘-选项。在这种情况下，LLM 被分为两个分片。一个分配给边缘设备，另一个分配给云服务器。对于LLM的划分策略，我们也使用了所提出的动态规划算法。不同的是，只有两个设备作为算法输入。

B. Overall Evaluation

我们将AGX Orin设置为源节点，源节点与云服务器之间的带宽为1Mbps。其他计算设备之间的带宽设置为50Mbps，差异为20%。为了测试吞吐量，我们将批量大小设置为参与设备可以支持的最大批量大小。LLM 推理的延迟和吞吐量如表所示。四。

我们有以下观察。首先，EdgeShard 对于大语言模型部署具有潜力且有益。对于 Llama2-70B 型约为

TABLE IV
PERFORMANCE OF LLM INFERENCE. (AVERAGE LATENCY: MILLISECONDS/TOKEN; THROUGHPUT: TOKENS/SECOND).

	Llama2-7B		Llama2-13B		Llama2-70B	
	Latency	Throughput	Latency	Throughput	Latency	Throughput
Edge-Solo	140.34	24.36	OOM	OOM	OOM	OOM
Cloud-Edge-Even	227.35	7.56	319.44	4.68	OOM	OOM
Cloud-Edge-Opt	140.34	24.36	243.45	4.74	OOM	OOM
EdgeShard	75.88	52.45	173.43	10.45	3086.43	1.25

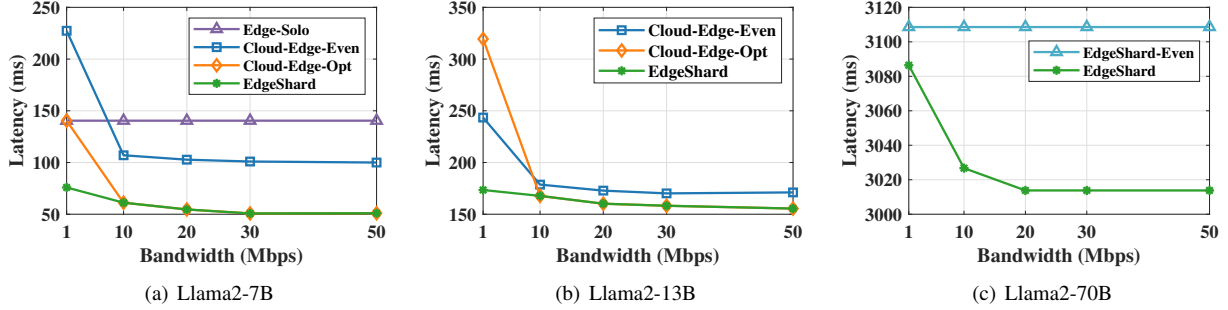


Fig. 7. Impact of Network Bandwidth to Latency of Collaborative LLMs inference

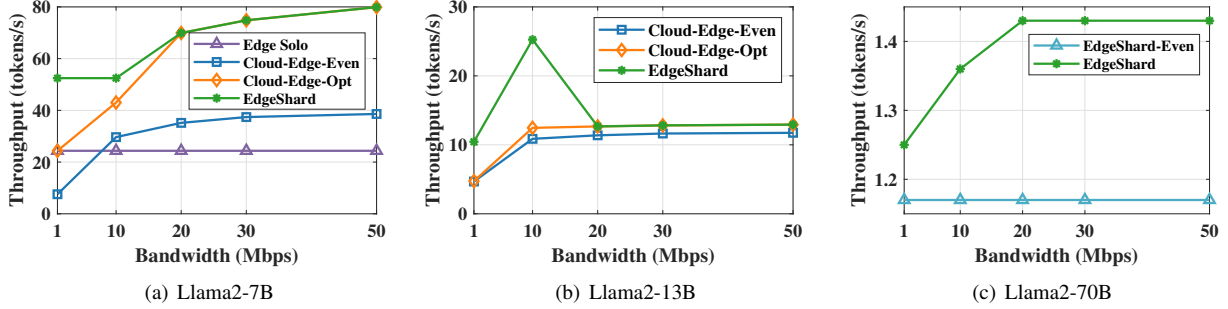


Fig. 8. Impact of Bandwidth to Throughput of Collaborative LLMs Inference

280GB, which far exceeds the memory capacity of solo edge deployment and cloud-edge collaborative deployment. They will have the out-of-memory issue (OOM). However, EdgeShard tackles this challenge by splitting the large model into shards and allocating them to multiple devices, enabling collaborative model inference. Second, EdgeShard achieves obviously lower inference latency and higher inference throughput than baseline methods. For Llama2-7B model, EdgeShard achieves 75.88ms latency, which is about 1.85x faster than Edge-Solo and Cloud-Edge-Opt, and about 3x faster than Cloud-Edge-Even. For the inference throughput, EdgeShard achieves 52.45 tokens per second with a maximum batch size of 8, which is around 2.2 times larger than Edge-Solo and Cloud-Edge-Opt, and about 7 times larger than Cloud-Edge-Even. Similar performance improvement is also observed for Llama2-13B model, where EdgeShard achieves 45.7% and 28.8% lower latency than Cloud-Edge-Even and Cloud-Edge-Opt, respectively. Also, EdgeShard has 2.23x and 2.2x higher throughput than Cloud-Edge-Even and Cloud-Edge-Opt. Third, we can also see that, for Llama2-7B, Cloud-Edge-Opt tends to have the same performance in terms of both inference latency and throughput as Edge-Solo. This is because the bandwidth between the source node and the cloud server is very limited in this experimental setting, i.e., 1Mbps. The

optimal deployment strategy of Cloud-Edge-Collaboration is local execution, which is the same as Edge-Solo.

C. Effects of Bandwidth

We set the source node as AGX Orin and vary the bandwidth between the cloud server and the source node from 1Mbps to 50Mbps. The performance of the latency and throughput of LLM inference are shown in Fig. 7 and Fig. 8, respectively.

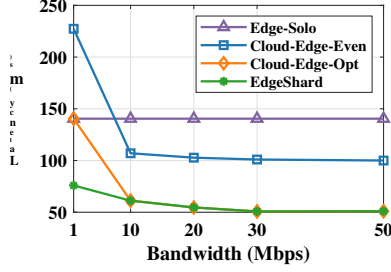
For Llama2-13B, a single AGX Orin cannot accommodate the full model. We only compare the performance among Cloud-Edge-Even, Cloud-Edge-Opt, and EdgeShard. Similarly, due to the memory constraint, the three baseline methods are not able to deploy the Llama2-70B model. Instead, we compare the performance of EdgeShard with its variant, i.e., EdgeShard-Even, where the model is equally partitioned and deployed to all the participating computing devices. It selects 11 AGX Orin and 1 RTX 3090 to deploy the Llama2-70B model.

In terms of latency, except for Edge-Solo, the latency of the other three methods decreases with the increasing bandwidth. This is because the three methods are collaboration-based, and the latency is influenced by the data transmission time. The increasing bandwidth leads to reduced communication

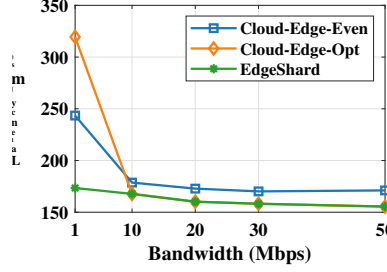
牌/秒)。

表 IV LLM 推理的性能。(平均延迟: 毫秒/令牌; 吞吐量: 令

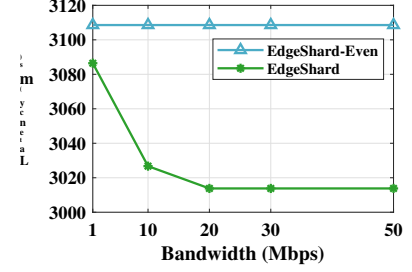
	Llama2-7B		Llama2-13B		Llama2-70B	
	Latency	Throughput	Latency	Throughput	Latency	Throughput
Edge-Solo	140.34	24.36	OOM	OOM	OOM	OOM
Cloud-Edge-Even	227.35	7.56	319.44	4.68	OOM	OOM
Cloud-Edge-Opt	140.34	24.36	243.45	4.74	OOM	OOM
EdgeShard	75.88	52.45	173.43	10.45	3086.43	1.25



(a) Llama2-7B



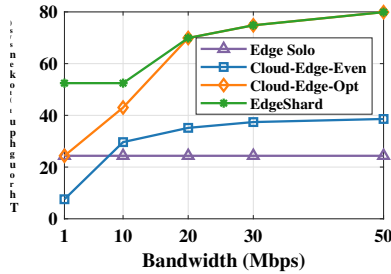
(b) Llama2-13B



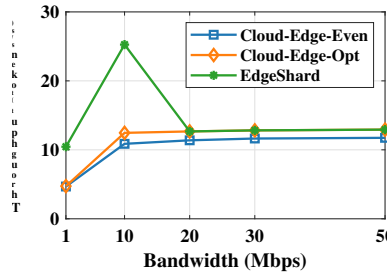
(c) Llama2-70B

如图7. 网络带宽对协作 LLM 延迟的影响推断

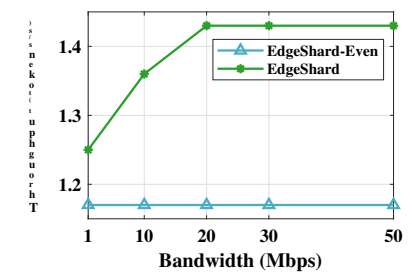
恩斯



(a) Llama2-7B



(b) Llama2-13B



(c) Llama2-70B

图 8. 带宽对协作 LLM 推理吞吐量的影响

280GB, 远超单边部署和云边协同部署的内存容量。他们将遇到内存不足问题 (OOM)。然而, EdgeShard 通过将大型模型分割成碎片并将其分配给多个设备来解决这一挑战, 从而实现协作模型推理。其次, 与基线方法相比, EdgeShard 明显实现了更低的推理延迟和更高的推理吞吐量。对于 Llama2-7B 模型, EdgeShard 实现了 75.88 毫秒的延迟, 比 Edge-Solo 和 Cloud-Edge-Opt 快约 1.85 倍, 比 Cloud-Edge-Even 快约 3 倍。对于推理吞吐量, EdgeShard 可实现每秒 52.45 个令牌, 最大批量大小为 8, 大约是 Edge-Solo 和 Cloud-Edge-Opt 的 2.2 倍, 是 Cloud-Edge-Even 的大约 7 倍。Llama2-13B 模型也观察到类似的性能改进, 其中 EdgeShard 的延迟分别比 Cloud-Edge-Even 和 Cloud-Edge-Opt 降低 45.7% 和 28.8%。此外, EdgeShard 的吞吐量比 Cloud-Edge-Even 和 Cloud-Edge-Opt 高 2.23 倍和 2.2 倍。第三, 我们还可以看到, 对于 Llama2-7B, Cloud-Edge-Opt 在推理延迟和吞吐量方面往往具有与 Edge-Solo 相同的性能。这是因为在本实验设置中源节点和云服务器之间的带宽非常有限, 即 1Mbps。这

Cloud-Edge-Collaboration 的最佳部署策略是本地执行, 与 Edge-Solo 相同。

C. Effects of Bandwidth

我们将源节点设置为 AGX Orin, 并将云服务器和源节点之间的带宽从 1Mbps 更改为 50Mbps。LLM 推理的延迟和吞吐量的性能分别如图 7 和图 8 所示。

对于 Llama2-13B, 单个 AGX Orin 无法容纳完整模型。我们仅比较 Cloud-Edge-Even、Cloud-Edge-Opt 和 EdgeShard 之间的性能。同样, 由于内存限制, 三种基线方法无法部署 Llama2-70B 模型。相反, 我们将 EdgeShard 与其变体 (即 EdgeShard-Even) 的性能进行比较, 其中模型被平均分区并部署到所有参与的计算设备。它选择 11 个 AGX Orin 和 1 个 RTX 3090 来部署 Llama2-70B 模型。

延迟方面, 除 Edge-Solo 外, 其他三种方法的延迟均随着带宽的增加而降低。这是因为这三种方法都是基于协作的, 并且延迟受到数据传输时间的影响。带宽增加导致通信减少

time. We can also see that for the collaboration methods, there is a dramatically latency reduction when the cloud-source bandwidth changes from 1Mbps to 10Mbps and a minor variance from 10Mbps to 50Mbps. This is because the bandwidth is gradually saturated at that time, and the computation time becomes the bottleneck.

Moreover, we can see that when the bandwidth is greater than 10Mbps, cloud-edge collaboration methods outperform the Edge-Solo method, as the cloud-edge collaboration methods introduce the powerful cloud server for computation acceleration. However, when the bandwidth is 1Mbps, Cloud-Edge-Even performs worse than EdgeSolo. This is because the data transmission cost is high in this case. The Cloud-Edge-Opt method tends to deploy the LLM model locally, which is the same as the Edge-Solo method. Interestingly, the latency of Cloud-Edge-Opt and EdgeShard is nearly the same when the bandwidth is greater than 10Mbps. We found that EdgeShard generates the same model partition and allocation policies as the Cloud-Edge-Opt method. The variance comes from the small fluctuations in model execution. It shows that the performance of EdgeShard will not be worse than that of Cloud-Edge-Opt, and the Cloud-Edge-Opt method is a special case of EdgeShard. A similar pattern is also observed for Llama2-13B. For Llama2-70B, EdgeShard performs better than its variant EdgeShard-Even, as there is resource heterogeneity among cloud server and edge devices, and EdgeShard adaptively partitions the LLMs among computing devices. However, the performance improvement is not so obvious as there are 11 AGX with the same computation capacity and only 1 RTX 3090.

In terms of throughput, similar patterns to the latency evaluation are also found for Llama2-7B model. Differently and interestingly, for Llama2-13B, EdgeShard does not show a closing performance with the Cloud-Edge-Opt method when the bandwidth is 10Mbps, but with a great improvement, where EdgeShard has about 2x higher throughput than the Cloud-Edge-Opt method. This is because of the high memory consumption of the RTX 3090 and the source node, i.e., AGX Orin. We observed that for the Cloud-Edge-Opt, the memory consumption of the two devices goes up to 95% and 98%, respectively, which only allows for a maximum batch size of 4. Otherwise, there will not be enough memory for the KV cache on the computing devices. However, when the bandwidth is 10Mbps, EdgeShard involves several edge devices where the memory consumption of an individual device becomes dramatically decreased, allowing for a larger batch size, i.e., 8 in this case. When the bandwidth is higher than 10Mbps, EdgeShards tends to have the same model partition and allocation strategy as Cloud-Edge-Opt, which yields a closing performance, as shown in Llama2-7B. For Llama2-70B, there is a slight throughput improvement of EdgeShard, and EdgeShard-Even shows a steady throughput as the evenly partition strategy will not change with the cloud-source bandwidth.

D. Effects of Source Node

We also test the influence of the source node on the inference latency and throughput, as the source node may have

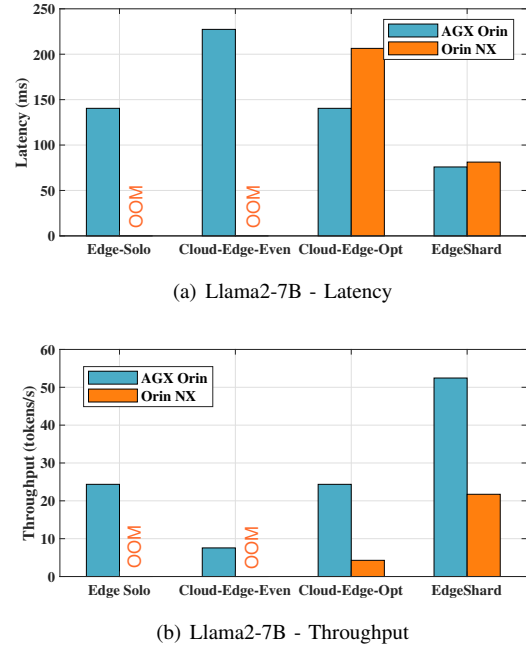


Fig. 9. Impact of Source Node

different computation and memory capacities, and EdgeShard enforces the first layer of LLM models residing on the source node to avoid raw data transmission. We set the source node as AGX Orin and Orin NX, respectively, and compare their performance. We set the bandwidth between the source node and the cloud server as 1Mbps. The results of Llama2-7B inference are shown in Fig. 9.

We find that when the source node is Orin NX, the Edge-Solo and Cloud-Edge-Even methods encounter the OOM error. This is due to the relatively lower memory of Orin NX, which cannot accommodate the Llama2-7B model, even for half part of the model. The difference between the two cases under the Cloud-Edge-Opt method is much more obvious than that of EdgeShard. For Cloud-Edge-Opt, there is about a 60ms gap, and for EdgeShard, the gap is about 5ms. This is because there are only two devices in the Cloud-Edge-Opt case, and it tends to put more layers on the source node. However, AGX Orin is much more powerful than Orin NX in terms of computation capacity. EdgeShard tends to involve more devices and put fewer model layers on the source node, which can fill in the gap in computation capacity between the source nodes. A similar phenomenon is also observed for the throughput, where AGX Orin has 6x higher throughput than Orin Nx for the Cloud-Edge-Opt method and only 2x higher throughput under the EdgeShard method. It shows EdgeShard can make full use of the computation resources in the network to optimize the performance.

E. Effects of Pipeline Execution strategy

We evaluate the two pipeline execution strategies. We set the bandwidth between the cloud server and the source node as 1Mbps. The results are shown in Fig. 10.

We can see that for all methods, EdgeShard-No-bubble outperforms EdgeShard-Bubble. Specifically, for Llama2-7b,

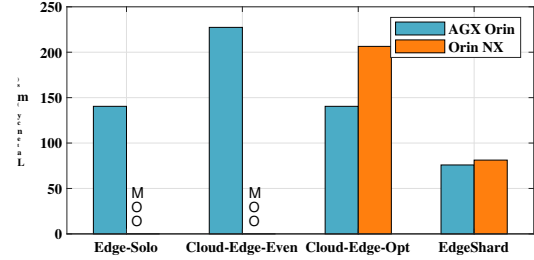
时间。我们还可以看到，对于协作方法，当云源带宽从 1 Mbps 变为 10Mbps 时，延迟显著降低，从 10Mbps 变为 50Mbps 时延迟也有微小变化。这是因为此时带宽逐渐饱和，计算时间成为瓶颈。

此外，我们可以看到，当带宽大于10Mbps时，云边协作方法优于Edge-Solo方法，因为云边协作方法引入了强大的云服务器进行计算加速。然而，当带宽为1Mbps时，Cloud-Edge-Even的性能比EdgeSolo差。这是因为这种情况下数据传输成本很高。Cloud-Edge-Opt方法倾向于在本地部署LLM模型，这与Edge-Solo方法相同。有趣的是，当带宽大于10Mbps时，Cloud-Edge-Opt和EdgeShard的延迟几乎相同。我们发现 EdgeShard 生成与 Cloud-Edge-Opt 方法相同的模型分区和分配策略。方差来自于模型执行的微小波动。说明EdgeShard的性能不会比Cloud-Edge-Opt差，而Cloud-Edge-Opt方法是EdgeShard的一个特例。Llama2-13B 也观察到类似的模式。对于 Llama2-70B，EdgeShard 的性能优于其变体 EdgeShard-Even，因为云服务器和边缘设备之间存在资源异构性，并且 EdgeShard 会在计算设备之间自适应地划分 LLM。不过，性能提升并不明显，因为相同计算能力的AGX有11个，而RTX 3090只有1个。

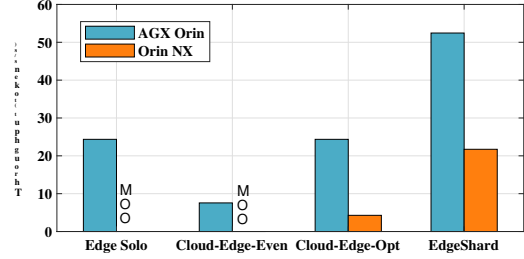
在吞吐量方面，Llama2-7B 模型也发现了与延迟评估类似的模式。不同且有趣的是，对于 Llama2-13B，EdgeShard 在带宽为 10Mbps 时并没有表现出与 Cloud-Edge-Opt 方法接近的性能，而是有了很大的改进，其中 EdgeShard 的吞吐量比 Cloud-Edge-Opt 方法高出约 2 倍。这是因为 RTX 3090 和源节点（即 AGX Orin）的内存消耗较高。我们观察到，对于Cloud-Edge-Opt，两个设备的内存消耗分别高达95%和98%，这仅允许最大batch size为4。否则，计算设备上将没有足够的内存用于KV缓存。然而，当带宽为 10Mbps 时，EdgeShard 涉及多个边缘设备，其中单个设备的内存消耗显著减少，从而允许更大的批量大小，即本例中的 8。当带宽高于10Mbps时，EdgeShards往往具有与Cloud-Edge-Opt相同的模型分区和分配策略，从而产生接近的性能，如Llama2-7B所示。对于Llama2-70B，EdgeShard的吞吐量略有提高，而EdgeShard-Even则表现出稳定的吞吐量，因为均匀分区策略不会随着云源带宽的变化而变化。

D. Effects of Source Node

我们还测试了源节点对推理延迟和吞吐量的影响，因为源节点可能有



(a) Llama2-7B - Latency



(b) Llama2-7B - Throughput

图 9. 源节点的影响

不同的计算和内存容量，EdgeShard 强制第一层 LLM 模型驻留在源节点上，以避免原始数据传输。我们分别将源节点设置为AGX Orin和Orin NX，并比较它们的性能。我们设置源节点和云服务器之间的带宽为1Mbps。Llama2-7B 推断结果如图9所示。

我们发现，当源节点为Orin NX时，Edge-Solo和Cloud-Edge-Even方法会遇到OOM错误。这是由于Orin NX的内存相对较低，无法容纳Llama2-7B模型，甚至无法容纳模型的一半。Cloud-Edge-Opt方式下两种情况的差异比EdgeShard方式明显得多。对于Cloud-Edge-Opt，大约有60ms的间隙，对于EdgeShard，间隙大约是5ms。这是因为 Cloud-Edge-Opt 情况下只有两台设备，并且往往会在源节点上放置更多层。不过，AGX Orin 在计算能力方面比 Orin NX 强大得多。EdgeShard往往会涉及更多的设备，并在源节点上放置更少的模型层，这可以填补源节点之间计算能力的差距。吞吐量方面也观察到类似的现象，其中 Cloud-Edge-Opt 方法下 AGX Orin 的吞吐量比 Orin Nx 高出 6 倍，而 EdgeShard 方法下的吞吐量仅高出 2 倍。这表明EdgeShard 可以充分利用网络中的计算资源来优化性能。

E. Effects of Pipeline Execution strategy

我们评估两种管道执行策略。我们将云服务器与源节点之间的带宽设置为1Mbps。结果如图10所示。

我们可以看到，对于所有方法，EdgeShard-No-bubble 都优于 EdgeShard-Bubble。具体来说，对于 Llama2-7b，

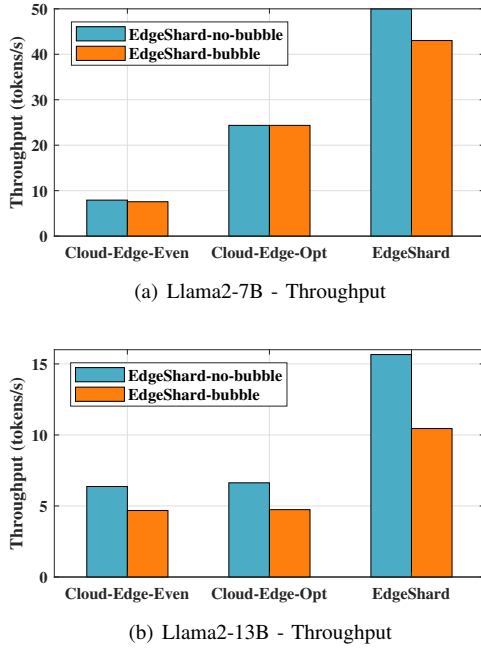


Fig. 10. Impact of Pipeline Execution Strategy

EdgeShard-No-bubble achieves an improvement of about 0.36 and 6.96 tokens per second than Edgeshard-bubble for Cloud-Edge-Even and EdgeShard, respectively. For the Cloud-Edge-Opt method, it selects local execution in this case. There is no pipeline execution, so the throughput for the two methods is the same. For Llama2-13b, EdgeShard-No-bubble achieves an improvement of about 1.69, 1.89, and 5.21 tokens per second than Edgeshard-Bubble for Cloud-Edge-Even, Cloud-Edge, and EdgeShard, respectively. Compared to EdgeShard-Bubble, EdgeShard-No-bubble does not need to wait for the completion of all micro-batches in an iteration and can effectively reduce the devices' idle time, thus leading to a higher throughput.

VI. RELATED WORK

This section reviews research works of LLM in the edge computing environment from two aspects, i.e., edge computing for efficient LLM deployment and LLM for optimizing edge computing.

A. Edge Computing for Efficient LLM

LLM is computation-intensive and memory-consuming. To address the issue of memory wall, quantization is widely adopted [7]–[12]. GPTQ [8] quantizes LLM with hundreds of billions of parameters to 3-4bits based on approximate second-order information. Lin et al. [10] reduce quantization error by optimizing channel scaling to preserve the salient important weights. They are weight-only quantization. SmoothQuant [11] and Agile-Quant [7] take a further step, which quantize not only the model weights, but also the model activations. However, the computation capacity and memory of a single device is still limited even for quantized LLM. Moreover, the performance of quantized LLM usually cannot be compared to that of its full-size model.

Other works [13], [14] tend to leverage the cloud-edge collaboration to partition and distribute the massive computation workload of LLM inference and finetuning. Wang et al. [13] increase the throughput by distributing the computation between cloud servers and edge devices, and reducing the communication overhead of transmitting the activations between the central cloud and edge devices by leveraging the low-rank property of residual activations. Chen et al. [14] efficiently leverage location-based information of edge devices for personalized prompt completion during collaborative edge-cloud LLM serving. However, the latency between edge devices and the central cloud is usually high and unstable, which will affect the inference and finetuning performance of LLM.

Our work is different from those works. We propose a general framework to integrate the computation resources of heterogeneous and ubiquitous cloud servers and edge devices. The framework allows the adaptive selection of computation devices and partition of the computation workload of LLM inference for optimized latency and throughput.

B. LLM for Optimizing Edge Computing

LLMs also have great potential in making complex and coherent decisions. There are also some works that leverage LLM to optimize resource utilization in edge computing, such as resource allocation and task offloading, network management, and intelligent IoT control. Li et al. [22] propose LAMBO, a LLM-based task offloading framework for mobile edge computing, to address the challenging issues of heterogeneous constraints, partial status perception, diverse optimization objectives, and dynamic environment that are not well addressed in traditional task offloading research. LAMBO shows that LLM is more effective compared to traditional DNN and deep reinforcement learning-based methods in complex and dynamic edge computing environments. They further design a LLM-based multi-agent system and incorporate communication knowledge and tools into the system, empowering it with the ability to optimize semantic communication in a 6G network [23]. Apart from optimization of resource utilization, Shen et al. [24] leverage the outstanding abilities of GPT in language understanding and code generation to train new models among federated edge devices. Rong et al. [25] leverage LLMs to generate adaptive control algorithms for addressing the diverse, dynamic, and decentralized network conditions in 6G integrated terrestrial network (TN) and non-terrestrial network (NTN). Though LLMs have shown great potential in making intelligent decisions, especially in complex and dynamic edge computing systems, the related research is still in the early stages. Challenges such as significant resource consumption, latency of decision-making, and uncertainty of generated decisions need further studies.

VII. DISCUSSION AND FUTURE WORKS

This section discusses some open issues and future works that may appeal to readers.

Incentive mechanisms. In this work, we partition the LLM into multiple shards and allocate them to heterogeneous devices. For edge computing scenarios, such as smart home

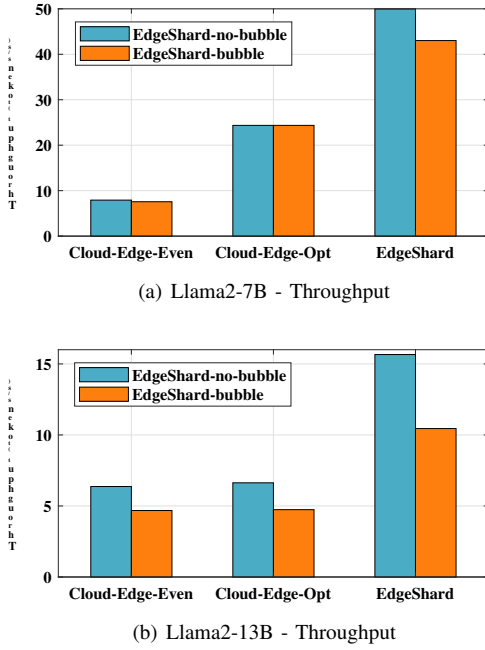


图 10. Pipeline 执行策略的影响

对于 Cloud-Edge-Even 和 EdgeShard, EdgeShard-No-bubble 分别比 Edgeshard-bubble 每秒提高约 0.36 和 6.96 个令牌。对于 Cloud-Edge-Opt 方法, 本例中选择本地执行。没有管道执行, 因此两种方法的吞吐量是相同的。对于 Llama 2-13b, EdgeShard-No-bubble 比 Cloud-Edge-Even、Cloud-Edge 和 EdgeShard 的 Edgeshard-Bubble 每秒分别提高了约 1.69、1.89 和 5.21 个令牌。与 EdgeShard-Bubble 相比, EdgeShard-No-bubble 不需要等待迭代中所有微批次完成, 可以有效减少设备的空闲时间, 从而带来更高的吞吐量。

六. 相关工作

本节从边缘计算高效部署 LLM 和优化边缘计算 LLM 两个方面回顾了 LLM 在边缘计算环境中的研究工作。

A. Edge Computing for Efficient LLM

LLM 是计算密集型且消耗内存的。为了解决内存墙问题, 量化被广泛采用[7]-[12]。GPTQ[8]基于近似二阶信息将具有数万亿参数的 LLM 量化到 3-4 位。林等人。[10]通过优化通道缩放来减少量化误差, 以保留显着的重要权重。它们是仅重量量化。SmoothQuant [11] 和 Agile-Quant [7] 更进一步, 不仅量化模型权重, 还量化模型激活。然而, 即使对于量化的 LLM, 单个设备的计算能力和内存仍然有限。此外, 量化 LLM 的性能通常无法与其全尺寸模型的性能进行比较。

其他工作[13]、[14]倾向于利用云边协作来划分和分配 LLM 推理和微调的大量计算工作量。王等人。[13]通过在云服务器和边缘设备之间分配计算来提高吞吐量, 并通过利用剩余激活的低秩属性来减少中央云和边缘设备之间传输激活的通信开销。陈等人。[14]有效地利用边缘设备的基于位置的信息, 在协作边缘云 LLM 服务期间实现个性化的提示完成。然而, 边缘设备和中央云之间的延迟通常很高且不稳定, 这会影响 LLM 的推理和微调性能。

我们的工作与那些工作不同。我们提出了一个通用框架来集成异构且无处不在的云服务器和边缘设备的计算资源。该框架允许自适应选择计算设备并划分 LLM 推理的计算工作负载, 以优化延迟和吞吐量。

B. LLM for Optimizing Edge Computing

大语言模型在做出复杂而连贯的决策方面也具有巨大潜力。还有一些工作利用 LLM 来优化边缘计算中的资源利用率, 例如资源分配和任务卸载、网络管理和智能物联网控制。李等人。[22]提出了 LAMBO, 一种基于 LLM 的移动边缘计算任务卸载框架, 以解决传统任务卸载研究中未能很好解决的异构约束、部分状态感知、多样化优化目标和动态环境等挑战性问题。LAMBO 表明, 在复杂和动态的边缘计算环境中, LLM 比传统的 DNN 和基于深度强化学习的方法更有效。他们进一步设计了一个基于 LLM 的多智能体系统, 并将通信知识和工具融入到系统中, 使其具有优化 6G 网络中语义通信的能力[23]。除了优化资源利用之外, Shen 等人。[24]利用 GPT 在语言理解和代码生成方面的出色能力来训练联合边缘设备之间的新模型。荣等人。[25]利用 LLM 生成自适应控制算法, 以解决 6G 集成地面网络 (TN) 和非地面网络 (NTN) 中多样化、动态和分散的网络条件。尽管大语言模型在智能决策方面表现出了巨大的潜力, 特别是在复杂和动态的边缘计算系统中, 但相关研究仍处于早期阶段。大量资源消耗、决策延迟以及生成决策的不确定性等挑战需要进一步研究。

七. 讨论和未来的工作

本节讨论一些可能吸引读者的悬而未决的问题和未来的工作。

激励机制。在这项工作中, 我们将 LLM 划分为多个片段并将它们分配给异构设备。针对边缘计算场景, 例如智能家居

and smart factory, there is a set of trusted devices owned by a single stakeholder. They may be able to use those devices for collaborative inference. However, if the devices belong to different stakeholders, they may not be willing to share devices' computation resources. Further incentive mechanisms are needed to reward resource sharing.

Batch size aware optimization. Large batch size will increase memory usage and affect the inference throughput. As shown in the experiment, by partitioning the workload of LLM inference to multiple devices, the memory usage of participating devices can be reduced and thus allows for a larger batch size, leading to increased throughput. However, the designed dynamic programming algorithm does not consider the influence of batch size, which remains space for further optimization.

VIII. CONCLUSION

In this work, we propose EdgeShard to enable the efficient deployment and distributed inference of LLMs on collaborative edge devices and cloud servers. We formulate a joint device selection and model partition problem to optimize inference latency and throughput, respectively, and solve it using dynamic programming algorithms. Experimental results show that edgesplit can adaptively determine the LLM partition and deployment strategy under various heterogeneous network conditions for optimizing inference performance. Edgeshard is not designed to replace cloud-based LLM inference, but to provide a flexible and adaptive LLM serving methods by utilizing ubiquitous computing devices. Experiments also shows that EdgeShard outperforms the cloud-edge collaborative inference method when cloud bandwidth is insufficient and tends to yield the same deployment strategy as the cloud-edge collaborative inference method when facing relatively abundant cloud bandwidth.

This is a pioneering work of deploying LLM in collaborative edge computing environment. We hope this work can stimulate more ideas and further research in this promising area.

IX. ACKNOWLEDGEMENT

This work was supported by the Research Institute for Artificial Intelligence of Things, The Hong Kong Polytechnic University, HK RGC Grant for Theme-based Research Scheme No. T43-513/23-N, and National Natural Science Foundation of China and Hong Kong RGC Collaborative Research Scheme No. CRS_PolyU501-23.

REFERENCES

- [1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, "Gpt-4 technical report," *arXiv preprint arXiv:2303.08774*, 2023.
- [2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, "Llama 2: Open foundation and fine-tuned chat models," *arXiv preprint arXiv:2307.09288*, 2023.
- [3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen *et al.*, "Palm 2 technical report," *arXiv preprint arXiv:2305.10403*, 2023.
- [4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," *Advances in neural information processing systems*, vol. 30, 2017.
- [5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, "Edge computing: Vision and challenges," *IEEE Internet of Things Journal*, vol. 3, no. 5, pp. 637–646, 2016.
- [6] J. Chen and X. Ran, "Deep learning with edge computing: A review," *Proceedings of the IEEE*, vol. 107, no. 8, pp. 1655–1674, 2019.
- [7] X. Shen, P. Dong, L. Lu, Z. Kong, Z. Li, M. Lin, C. Wu, and Y. Wang, "Agile-quant: Activation-guided quantization for faster inference of llms on the edge," *arXiv preprint arXiv:2312.05693*, 2023.
- [8] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "Gptq: Accurate post-training quantization for generative pre-trained transformers," *arXiv preprint arXiv:2210.17323*, 2022.
- [9] —, "Optq: Accurate quantization for generative pre-trained transformers," in *The Eleventh International Conference on Learning Representations*, 2022.
- [10] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang, and S. Han, "Awq: Activation-aware weight quantization for llm compression and acceleration," *arXiv preprint arXiv:2306.00978*, 2023.
- [11] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "Smoothquant: Accurate and efficient post-training quantization for large language models," in *International Conference on Machine Learning*. PMLR, 2023, pp. 38 087–38 099.
- [12] X. Shen, Z. Kong, C. Yang, Z. Han, L. Lu, P. Dong, C. Lyu, C.-h. Li, X. Guo, Z. Shu *et al.*, "Edgeqat: Entropy and distribution guided quantization-aware training for the acceleration of lightweight llms on the edge," *arXiv preprint arXiv:2402.10787*, 2024.
- [13] Y. Wang, Y. Lin, X. Zeng, and G. Zhang, "Privatelora for efficient privacy preserving llm," *arXiv preprint arXiv:2311.14030*, 2023.
- [14] Y. Chen, R. Li, Z. Zhao, C. Peng, J. Wu, E. Hossain, and H. Zhang, "Netgpt: A native-ai network architecture beyond provisioning personalized generative services," 2023.
- [15] M. Zhang, J. Cao, Y. Sahni, Q. Chen, S. Jiang, and T. Wu, "Eaas: A service-oriented edge computing framework towards distributed intelligence," in *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. IEEE, 2022, pp. 165–175.
- [16] M. Zhang, J. Cao, L. Yang, L. Zhang, Y. Sahni, and S. Jiang, "Ents: An edge-native task scheduling system for collaborative edge computing," in *2022 IEEE/ACM 7th Symposium on Edge Computing (SEC)*. IEEE, 2022, pp. 149–161.
- [17] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu *et al.*, "Gpipe: Efficient training of giant neural networks using pipeline parallelism," *Advances in neural information processing systems*, vol. 32, 2019.
- [18] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons, and M. Zaharia, "Pipedream: Generalized pipeline parallelism for dnn training," in *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019, pp. 1–15.
- [19] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," *arXiv preprint arXiv:1810.04805*, 2018.
- [20] B. Hubert *et al.*, "Linux advanced routing & traffic control howto," *Netherlabs BV*, vol. 1, pp. 99–107, 2002.
- [21] S. Merity, C. Xiong, J. Bradbury, and R. Socher, "Pointer sentinel mixture models," in *International Conference on Learning Representations*, 2017.
- [22] L. Dong, F. Jiang, Y. Peng, K. Wang, K. Yang, C. Pan, and R. Schober, "Lambo: Large language model empowered edge intelligence," *arXiv preprint arXiv:2308.15078*, 2023.
- [23] F. Jiang, L. Dong, Y. Peng, K. Wang, K. Yang, C. Pan, D. Niyato, and O. A. Dobre, "Large language model enhanced multi-agent systems for 6g communications," *arXiv preprint arXiv:2312.07850*, 2023.
- [24] Y. Shen, J. Shao, X. Zhang, Z. Lin, H. Pan, D. Li, J. Zhang, and K. B. Letaief, "Large language models empowered autonomous edge ai for connected intelligence," *IEEE Communications Magazine*, 2024.
- [25] B. Rong and H. Rutagwema, "Leveraging large language models for intelligent control of 6g integrated tn-ntn with iot service," *IEEE Network*, 2024.

和智能工厂一样，有一组由单个利益相关者拥有的可信设备。他们也许能够使用这些设备进行协作推理。然而，如果设备属于不同的利益相关者，他们可能不愿意共享设备的计算资源。需要进一步的激励机制来奖励资源共享。

批量大小感知优化。大批量会增加内存使用并影响推理吞吐量。如实验所示，通过将LLM推理的工作负载划分到多个设备，可以减少参与设备的内存使用量，从而允许更大的批量大小，从而提高吞吐量。然而，设计的动态规划算法没有考虑批量大小的影响，这仍有进一步优化的空间。

八. 结论

在这项工作中，我们提出 EdgeShard 来实现 LLM 在协作边缘设备和云服务器上的高效部署和分布式推理。我们制定联合设备选择和模型分区问题来分别优化推理延迟和吞吐量，并使用动态编程算法来解决它。实验结果表明，edgesplit可以在各种异构网络条件下自适应地确定LLM分区和部署策略，以优化推理性能。Edgeshard并不是为了取代基于云的LLM推理，而是利用普适的计算设备提供灵活、自适应的LLM服务方法。实验还表明，当云带宽不足时，EdgeShard 的性能优于云边协同推理方法；而当云带宽相对充裕时，EdgeShard 往往会产生与云边协同推理方法相同的部署策略。

这是在协作边缘计算环境中部署LLM的开创性工作。我们希望这项工作能够激发这个有希望的领域的更多想法和进一步的研究。

九. 致谢

这项工作得到了香港理工大学物联网人工智能研究所、香港研究资助局主题研究计划（编号T43-513/23-N）和国家自然科学基金委员会与香港研究资助局合作研究计划（编号CRS PolyU501-23）的支持。

参考

[1] J. Achiam, S. Adler, S. Agarwal, L. Ahmad, I. Akkaya, F. L. Aleman, D. Almeida, J. Altschmidt, S. Altman, S. Anadkat *et al.*, “Gpt-4 技术报告”, *arXiv preprint arXiv:2303.08774*, 2023 年。[2] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: 开放基础和微调聊天模型”, *arXiv preprint arXiv:2307.09288*, 2023 年。[3] R. Anil, A. M. Dai, O. Firat, M. Johnson, D. Lepikhin, A. Passos, S. Shakeri, E. Taropa, P. Bailey, Z. Chen *et al.*, “Palm 2 技术报告”, *arXiv preprint arXiv:2305.10403*, 2023 年。[4] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, ̄. Kaiser 和 I. Polosukhin, “注意力就是你所需要的”, *Advances in neural information processing systems*, 卷。2017 年 30 日。

[5] W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “边缘计算：愿景与挑战”, *IEEE Internet of Things Journal*, 卷。3, 没有。5, 第 637-646 页, 2016 年。[6] J. Chen 和 X. Ran, “边缘计算的深度学习：综述”, *Proceedings of the IEEE*, 卷。107, 没有。8, 第 1655-1674 页, 2019 年。[7] X. Shen, P. Dong, L. Lu, Z. Kong, Z. Li, M. Lin, C. Wu 和 Y. Wang, “Agile-quant: 激活引导量化, 以实现边缘上 llms 的更快推理”, *arXiv preprint arXiv:2312.05693*, 2023 年。[8] E. Frantar, S. Ashkboos, T. Hoefler 和 D. Alistarh, “Gptq: 生成式预训练 Transformer 的准确后训练量化”, *arXiv preprint arXiv:2210.17323*, 2022 年。[9] —, “Optq: 生成式预训练 Transformer 的准确量化”, *The Eleventh International Conference on Learning Representations*, 2022 年。[10] J. Lin, J. Tang, H. Tang, S. Yang, X. Dang 和 S. Han, “Awq: 用于 llm 压缩和加速的激活感知权重量化”, *arXiv preprint arXiv:2306.00978*, 2023。[11] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth 和 S. Han, “Smoothquant: 大语言模型的准确高效的训练后量化”, 载于 *International Conference on Machine Learning*. PMLR, 2023, 第 38 087-38 099 页。[12] X. Shen, Z. Kong, C. Yang, Z. Han, L. Lu, P. Dong, C. Lyu, C.-h. Li, X. Guo, Z. Shu *et al.*, “Edgeqat: 熵和分布引导的量化感知训练, 用于加速边缘轻量级 llm”, *arXiv preprint arXiv:2402.10787*, 2024。[13] Y. Wang, Y. Lin, X. Zeng 和 G. Zhang, “用于高效隐私保护 llm 的 Privatelora”, *arXiv preprint arXiv:2311.14030*, 2023。[14] Y. Chen, R. Li, Z. Zhao, C. Peng, J. Wu, E. Hossain 和 H. Zhang, “Netgpt: 超越提供个性化生成服务的原生人工智能网络架构”, 2023 年。[15] M. Zhang, J. Cao, Y. Sahni, Q. Chen, S. Jiang 和 T. Wu, “Eaas: 面向分布式智能的服务导向边缘计算框架”, *2022 IEEE International Conference on Service-Oriented System Engineering (SOSE)*. IEEE, 2022 年, 第 165-175 页。[16] M. Zhang, J. Cao, L. Yang, L. Zhang, Y. Sahni 和 S. Jiang, “Ents: 用于协作边缘计算的边缘原生任务调度系统”, 参见 *2022 IEEE/ACM 7th Symposium on Edge Computing (SEC)*. IEEE, 2022 年, 第 149-161 页。[17] Y. Huang, Y. Cheng, A. Bapna, O. Firat, D. Chen, M. Chen, H. Lee, J. Ngiam, Q. V. Le, Y. Wu *et al.*, “Gpipe: 使用管道并行性对巨型神经网络进行高效训练”, *Advances in neural information processing systems*, 卷。2019 年 32 月。[18] D. Narayanan, A. Harlap, A. Phanishayee, V. Seshadri, N. R. Devanur, G. R. Ganger, P. B. Gibbons 和 M. Zaharia, “Pipedream: DNN 训练的广义管道并行性”, 载于 *Proceedings of the 27th ACM Symposium on Operating Systems Principles*, 2019 年, 第 1-15 页。[19] J. 德夫林, M.-W. Chang, K. Lee 和 K. Toutanova, “Bert: 用于语言理解的深度双向转换器的预训练”, *arXiv preprint arXiv:1810.04805*, 2018。[20] B. Hubert *et al.*, “Linux 高级路由和流量控制指南”, *Netherlabs BV*, 卷。1, 第 99-107 页, 2002 年。[21] S. Merity, C. Xiong, J. Bradbury 和 R. Socher, “指针哨兵混合模型”, in *International Conference on Learning Representations*, 2017。[22] L. Dong, F. Jiang, Y. Peng, K. Wang, K. Yang, C. Pan 和 R. Schober, “Lambo: 大语言模型赋能边缘智能”, *arXiv preprint arXiv:2308.15078*, 2023。[23] F. Jiang, L. Dong, Y. Peng, K. Wang, K. Yang, C. Pan, D. Niya to 和 O. A. Dobre, “用于 6G 通信的大语言模型增强型多智能体系统”, *arXiv preprint arXiv:2312.07850*, 2023。[24] Y. Shen, J. Shao, X. Zhang, Z. Lin, H. Pan, D. Li, J. 张和 K. B. Letaief, “大语言模型为互联智能提供自主边缘人工智能”, *IEEE Communications Magazine*, 2024 年。[25] B. Rong 和 H. Rutagemwa, “利用大语言模型对 6g 集成 t n-ntn 与物联网服务进行智能控制”, *IEEE Network*, 2024 年。